

Chapter 6 Functions of Combinational Logic

Outline

- Basic Adders
- Parallel Binary Adders
- Ripple Carry versus Look-Ahead Carry Adders
- Comparators
- Decoders
- Encoders
- Code Converters
- Multiplexers
- Demultiplexers
- Race-Hazard

6.1 Basic Adders

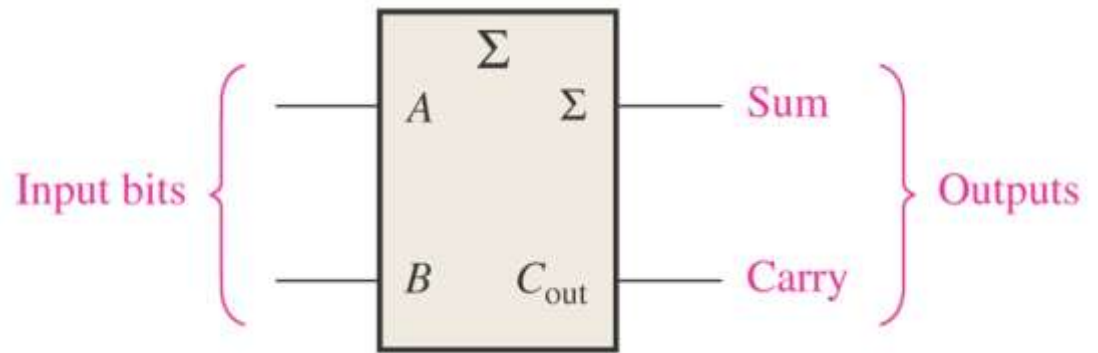
- The Half-Adder

$$0 + 0 = 0$$

$$0 + 1 = 1$$

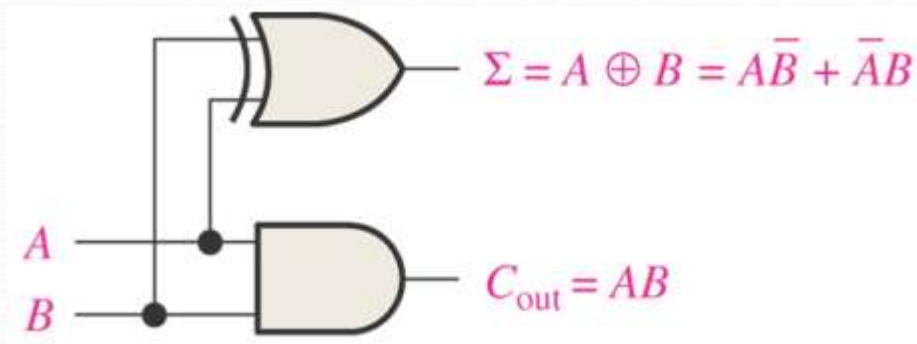
$$1 + 0 = 1$$

$$1 + 1 = 0$$



Truth table and logic diagram of half-adder

A	B	C_{out}	Σ
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



Full-adder

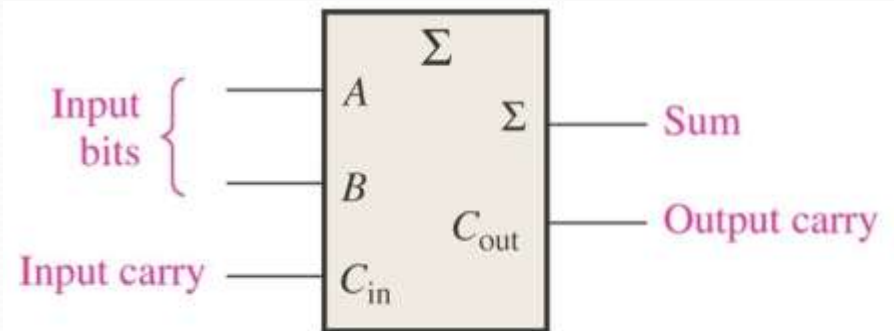
A	B	C _{in}	C _{out}	Σ
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$\Sigma = \overline{A}\overline{B}C_{in} + \overline{A}B\overline{C}_{in} + A\overline{B}\overline{C}_{in} + ABC_{in}$$

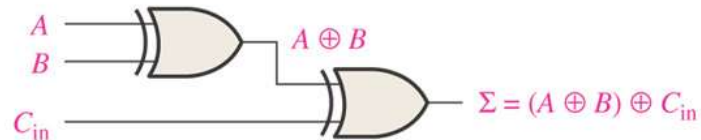
$$= (A \oplus B) \oplus C_{in}$$

$$C_{out} = \overline{A}BC_{in} + A\overline{B}C_{in} + AB\overline{C}_{in} + ABC_{in}$$

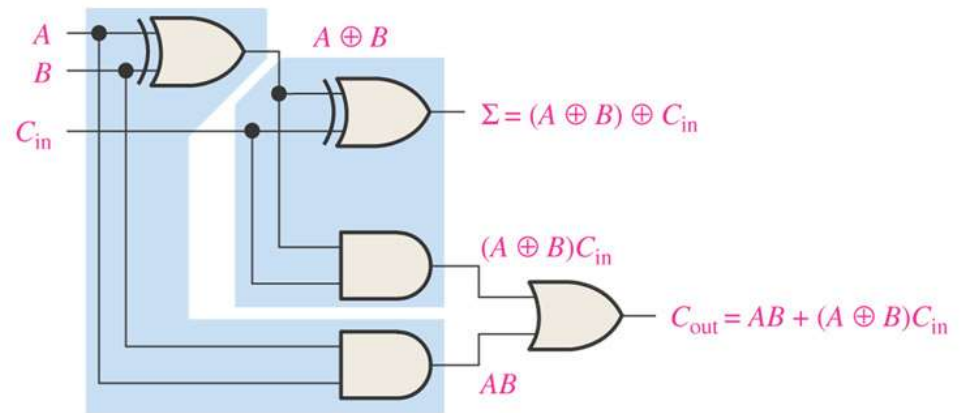
$$= AB + (A \oplus B)C_{in}$$



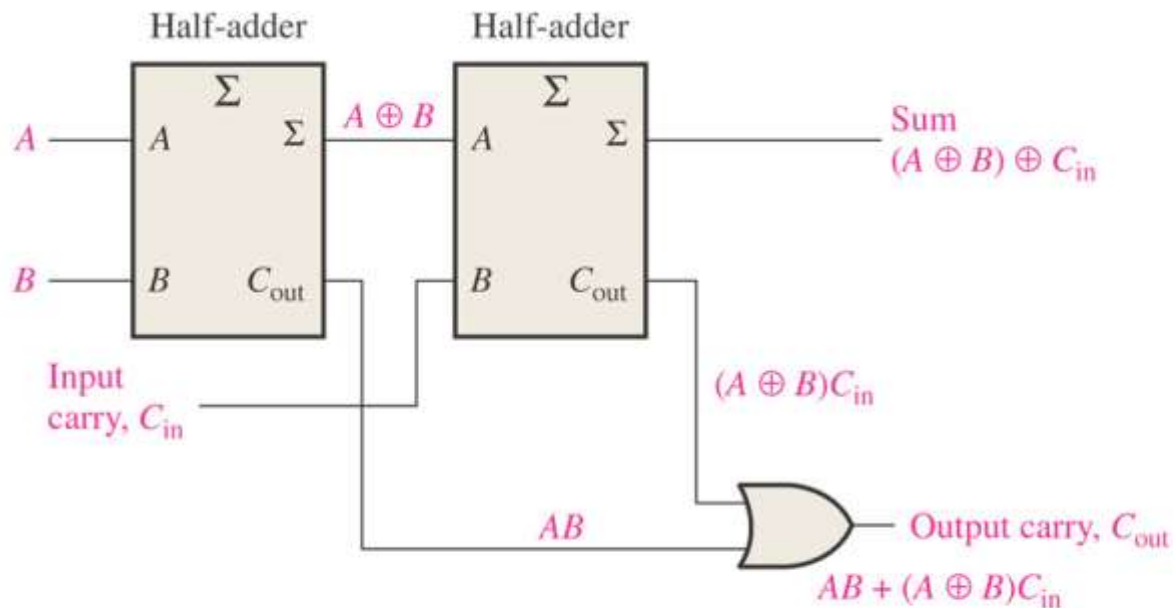
Full-adder logic and implementation



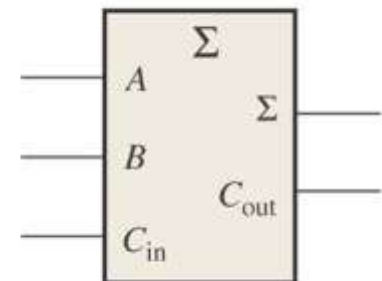
(a) Logic required to form the sum of three bits



(b) Complete logic circuit for a full-adder (each half-adder is enclosed by a shaded area)



(a) Arrangement of two half-adders to form a full-adder



(b) Full-adder logic symbol

6.2 Parallel Binary Adders

- To add binary numbers with more than one bit, additional full-adders must be used.
- The carry output of each adder is connected to the carry input of the next higher-order adder.

General format, addition
of two 2-bit numbers:

$$\begin{array}{r} A_2A_1 \\ + B_2B_1 \\ \hline \Sigma_3\Sigma_2\Sigma_1 \end{array}$$

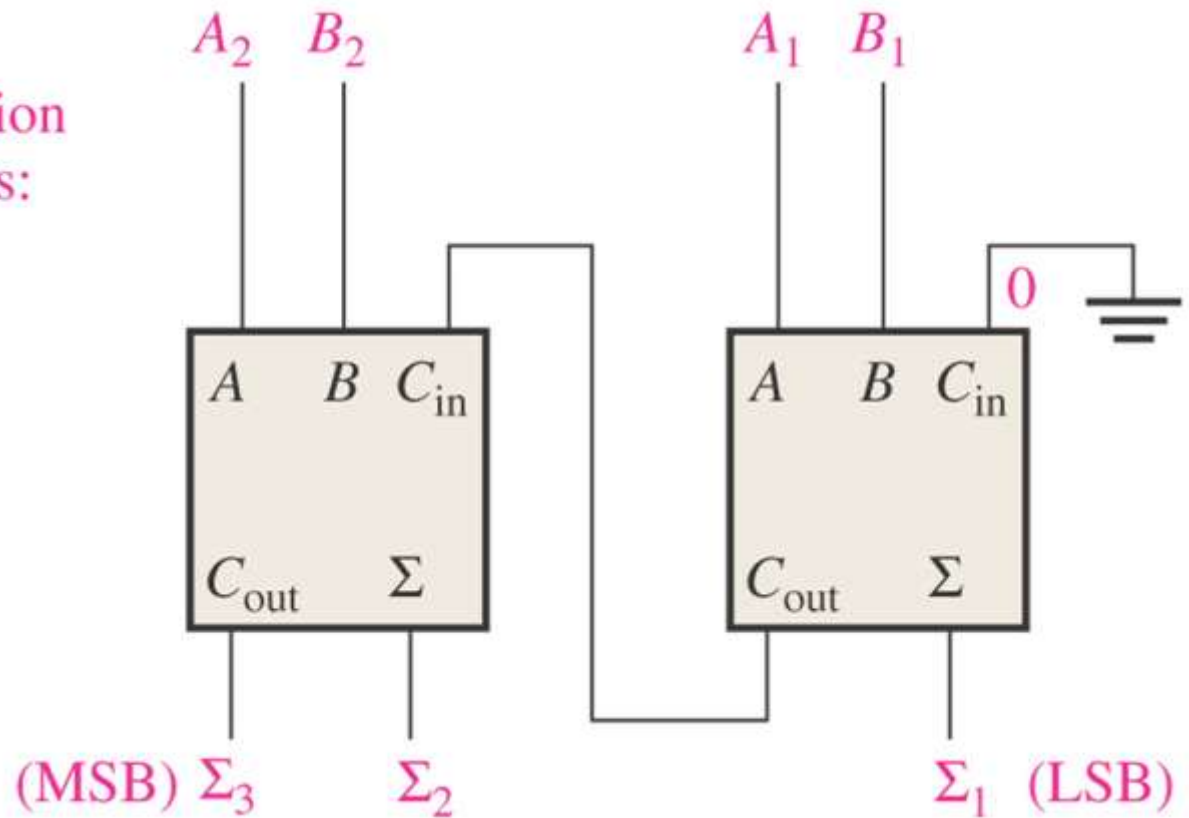
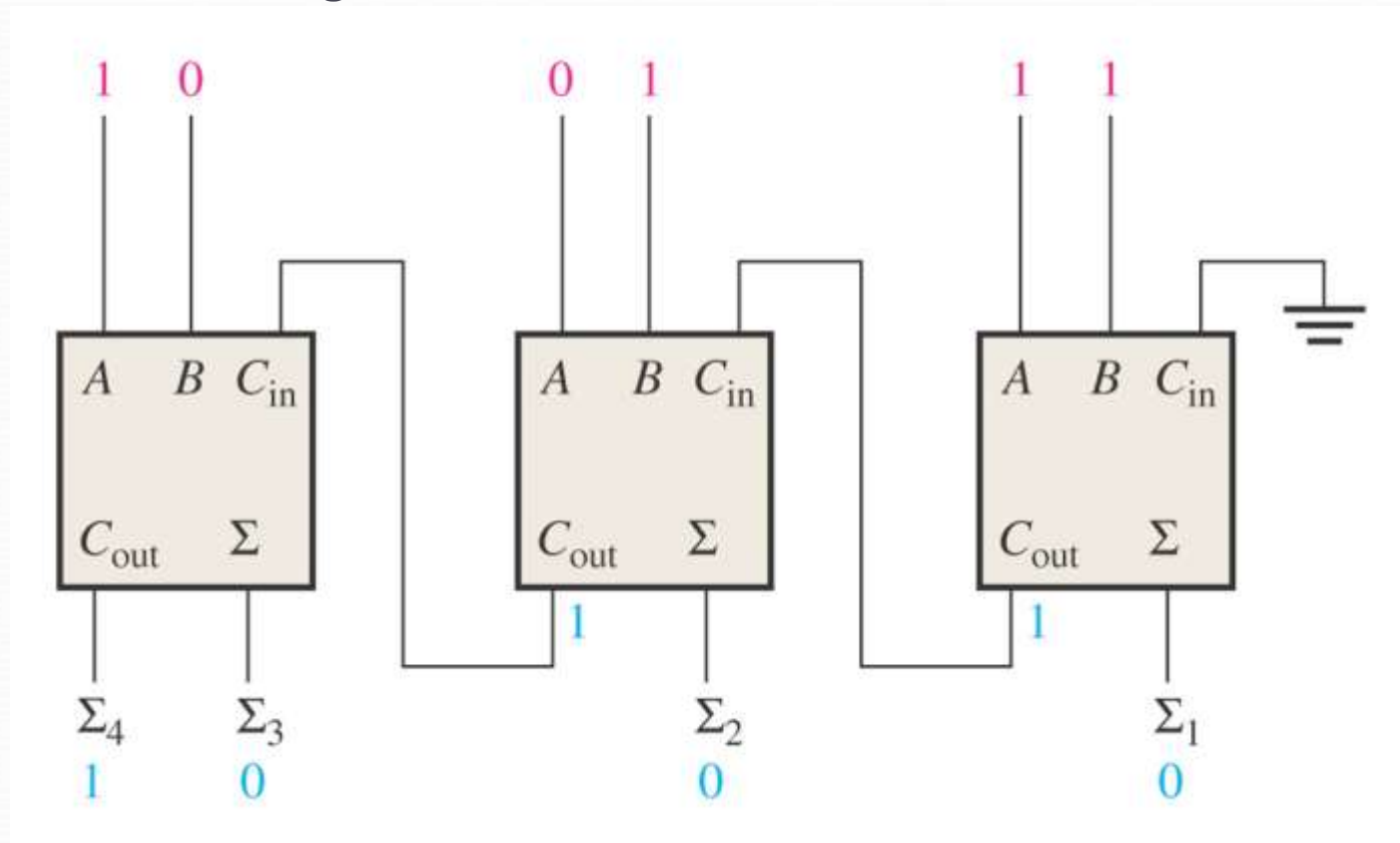
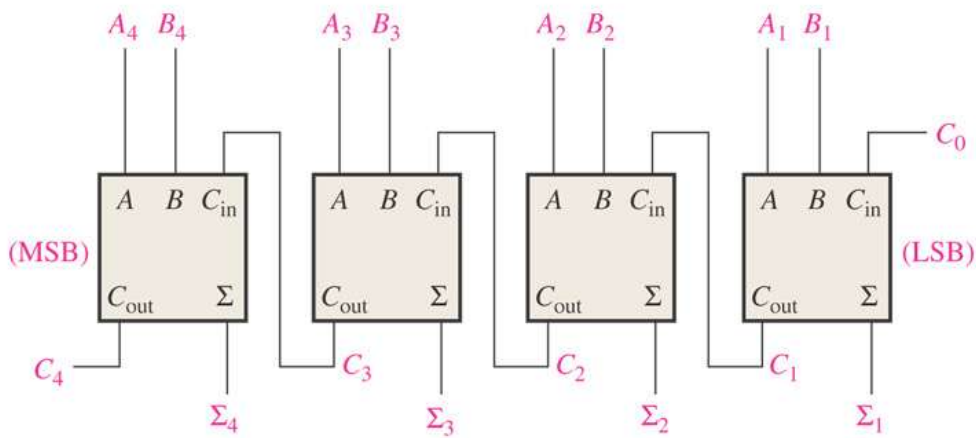


Figure 6–7 Block diagram of a basic 2-bit parallel adder using two full-adders

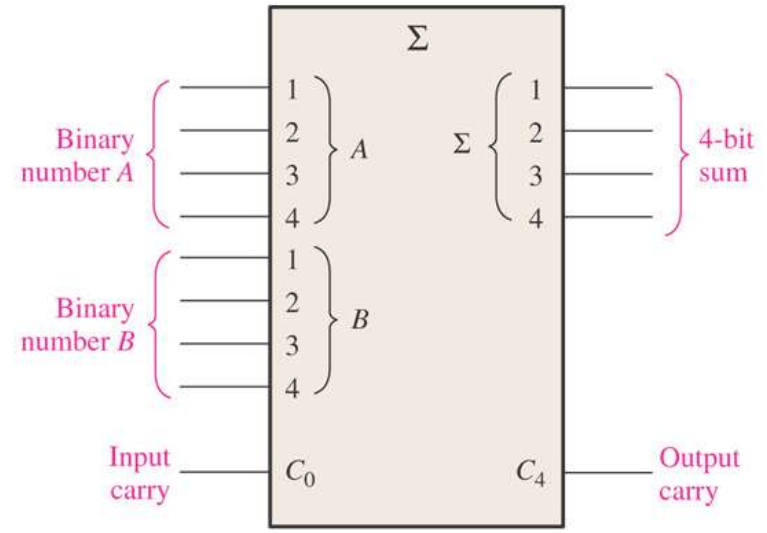
Figure 6–8 Determine the sum generated by the 3-bit parallel adder in the following figure and show the intermediate carries when the binary numbers 101 and 011 are being added.



Example: A 4-bit parallel adder.

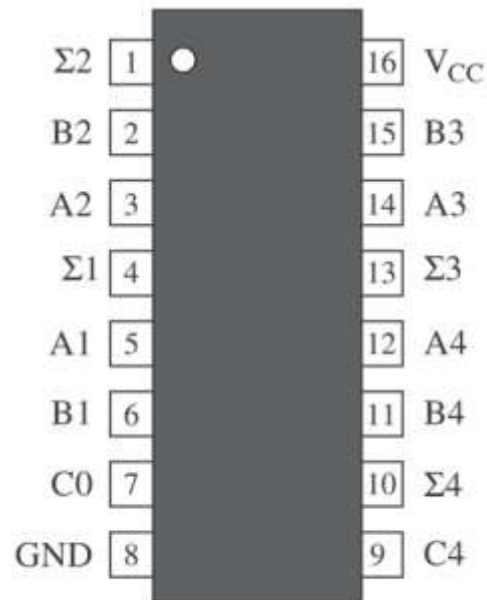


(a) Block diagram

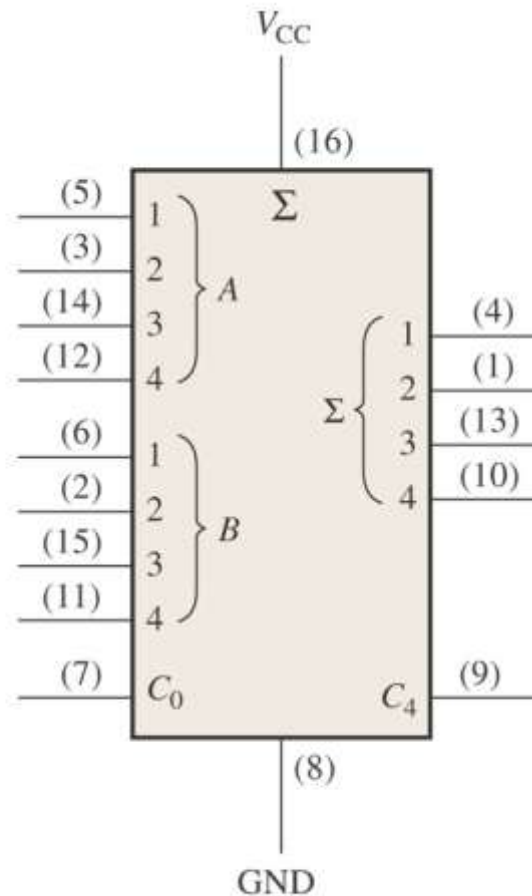


(b) Logic symbol

Example 74LS283 4-bit parallel adder



(a) Pin diagram of 74LS283

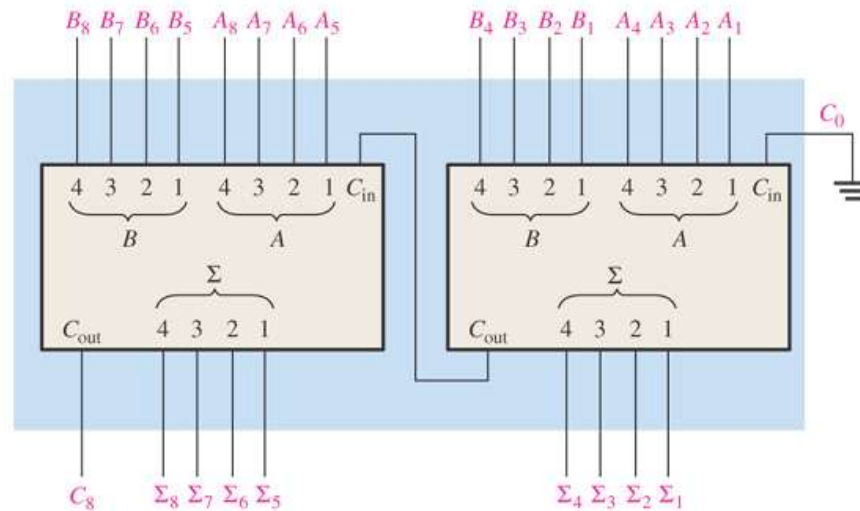


(b) 74LS283 logic symbol

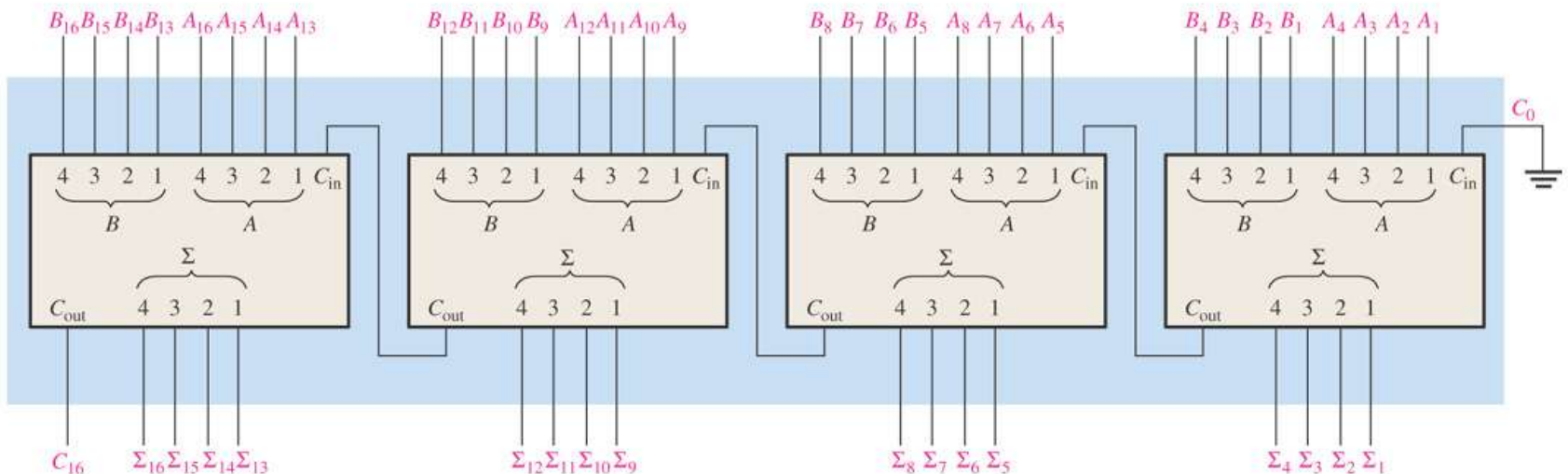
Figure 6–11 Propagation delay characteristics for the 74LS283.

Symbol	Parameter	Limits			Unit
		Min	Typ	Max	
t_{PLH} t_{PHL}	Propagation delay, C_0 input to any Σ output		16 15	24 24	ns
t_{PLH} t_{PHL}	Propagation delay, any A or B input to Σ outputs		15 15	24 24	ns
t_{PLH} t_{PHL}	Propagation delay, C_0 input to C_4 output		11 11	17 22	ns
t_{PLH} t_{PHL}	Propagation delay, any A or B input to C_4 output		11 12	17 17	ns

Examples: Adder expansion.



(a) Cascading of two 4-bit adders to form an 8-bit adder



(b) Cascading of four 4-bit adders to form a 16-bit adder

Example: Two 74LS283 adders connected as an 8-bit parallel adder (pin numbers are in parentheses).

$A_8A_7A_6A_5A_4A_3A_2A_1=10111001$, $B_8B_7B_6B_5B_4B_3B_2B_1=10011110$

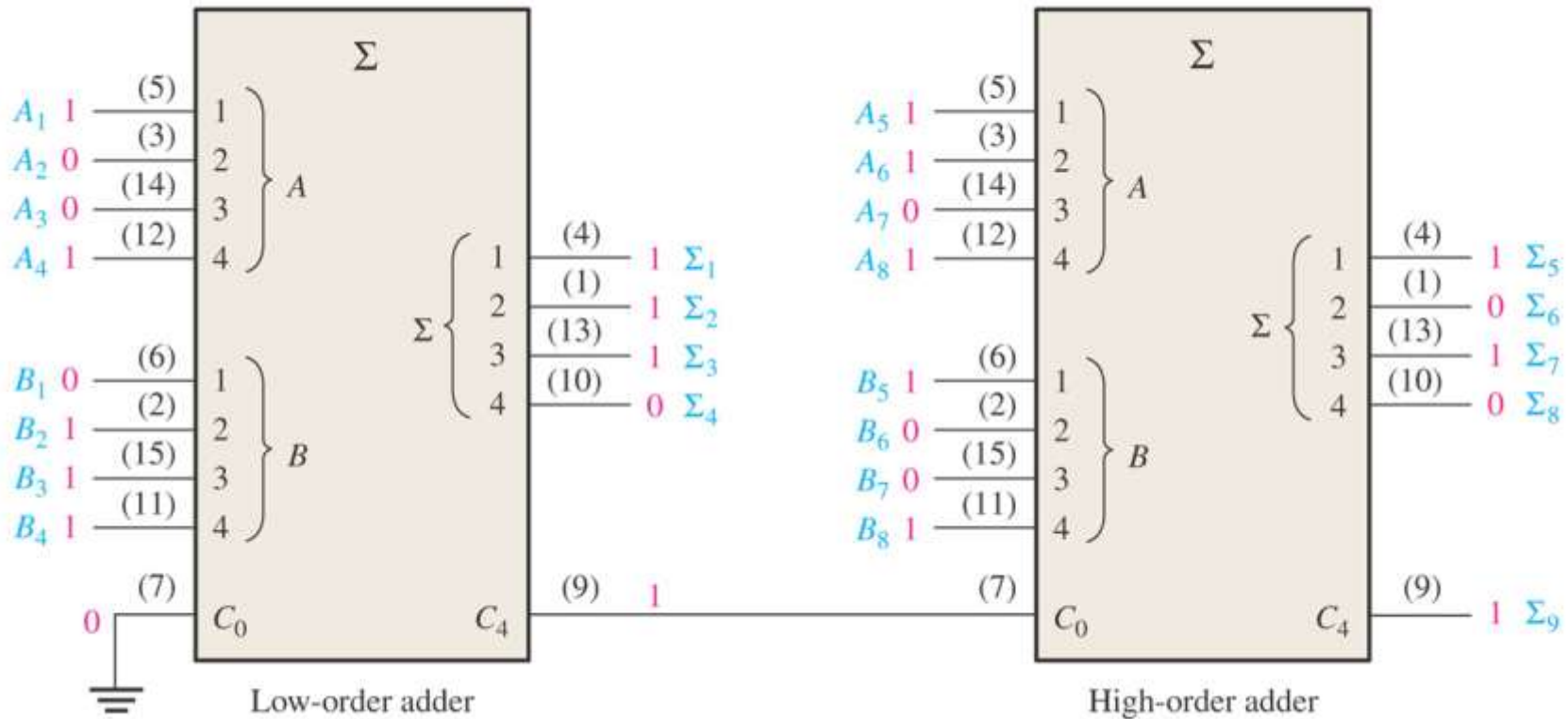
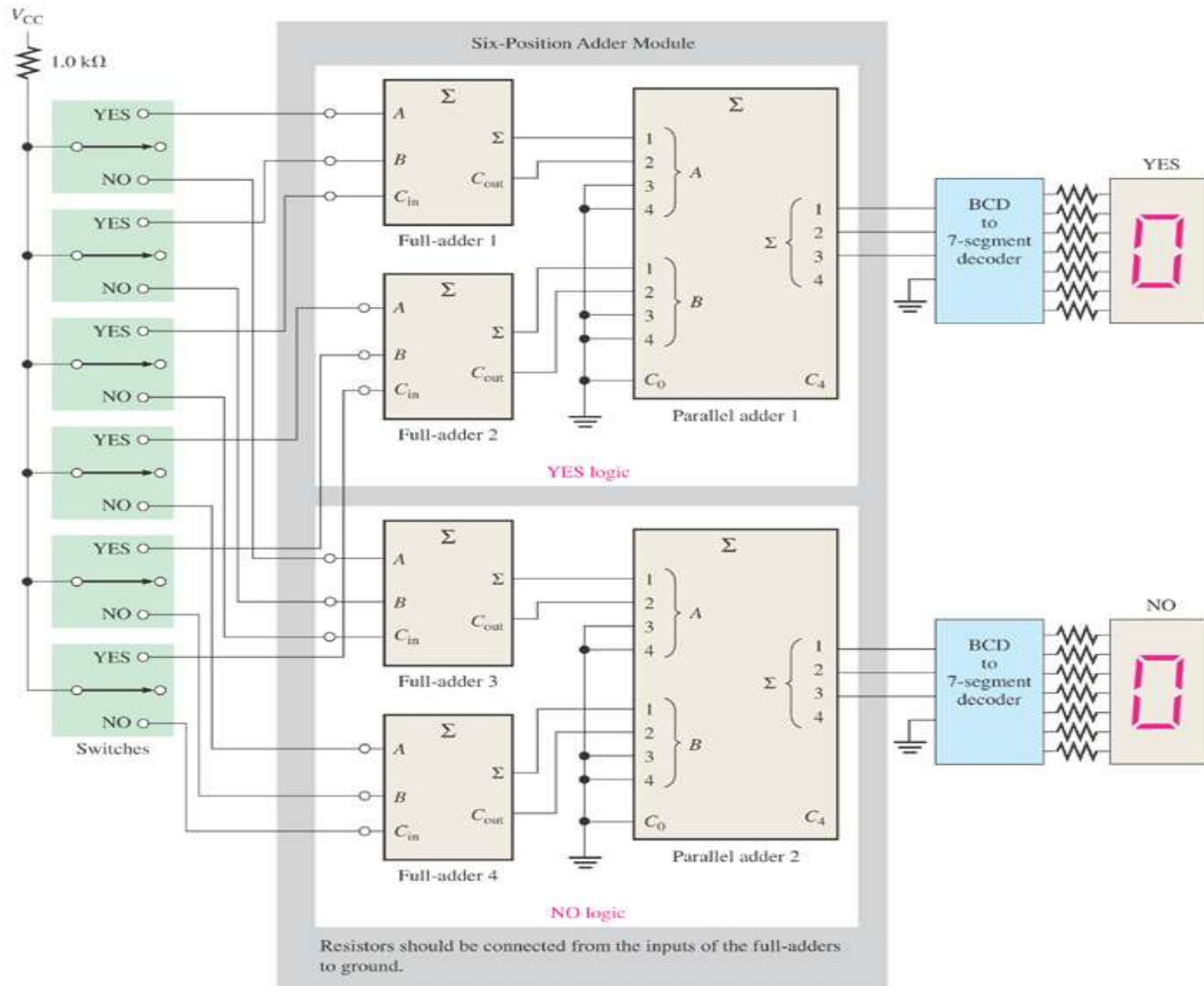


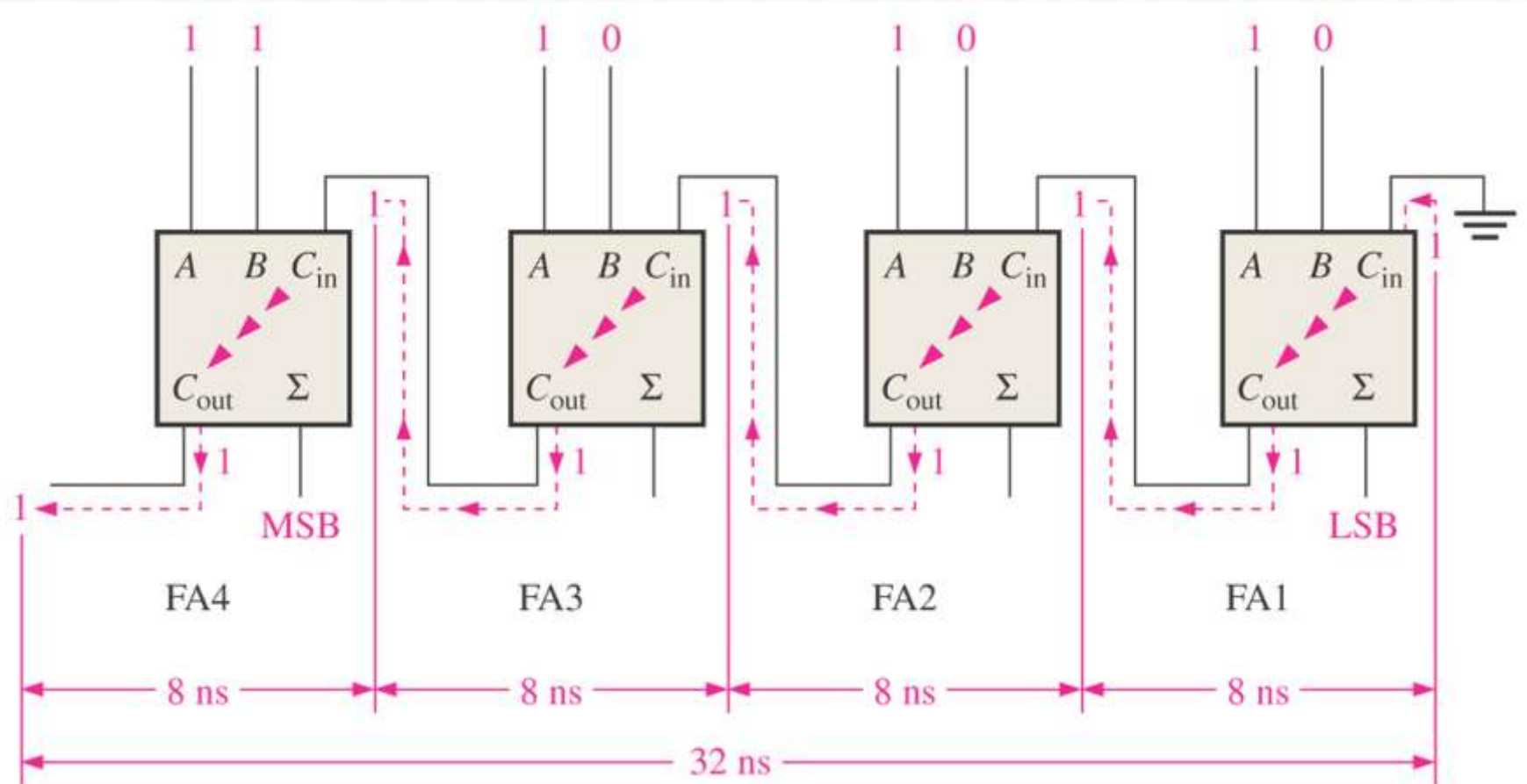
Figure 6–14 A voting system using full-adders and parallel binary adders.



6.3 Ripple Carry Versus Look-Ahead Carry Adders

- Ripple carry adder: the carry output of each full-adder is connected to the carry input of the next higher-order stage.
 - Time delay
- The look-ahead carry adder: anticipates the output carry of each stage, and based on the inputs, produces the output carry by either carry generation or carry propagation.

Figure 6–15 A 4-bit parallel ripple carry adder showing “worst-case” carry propagation delays.



• The look-ahead carry adder

- Carry generation

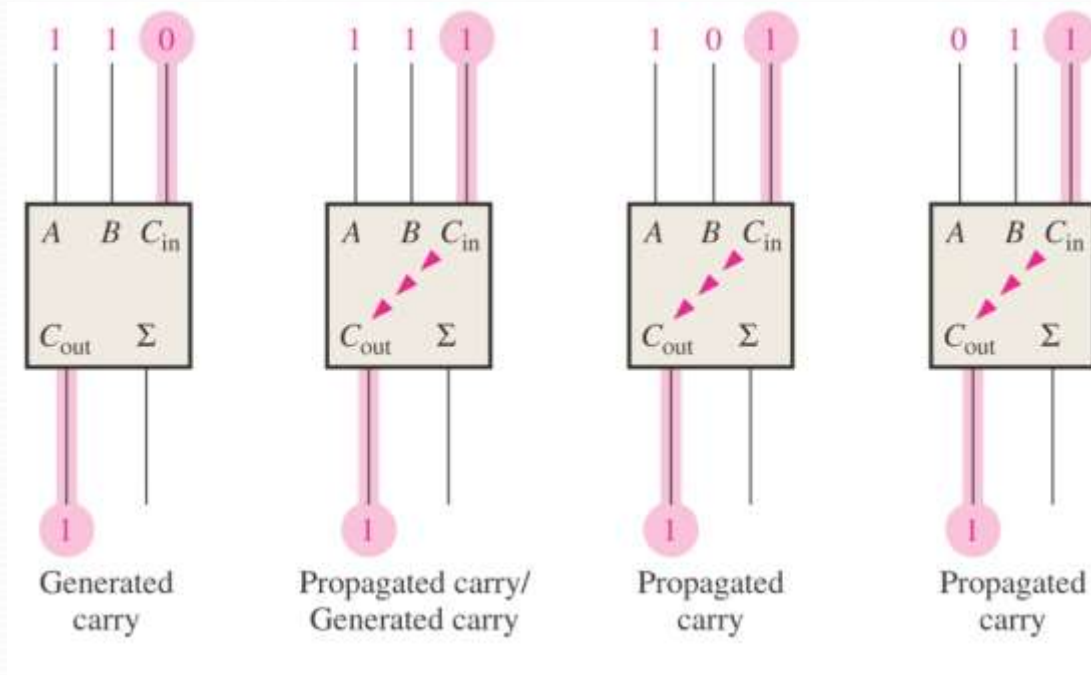
$$C_g = AB$$

- Carry propagation

$$C_p = A + B$$

- Output carry

$$C_{out} = C_g + C_p C_{in}$$



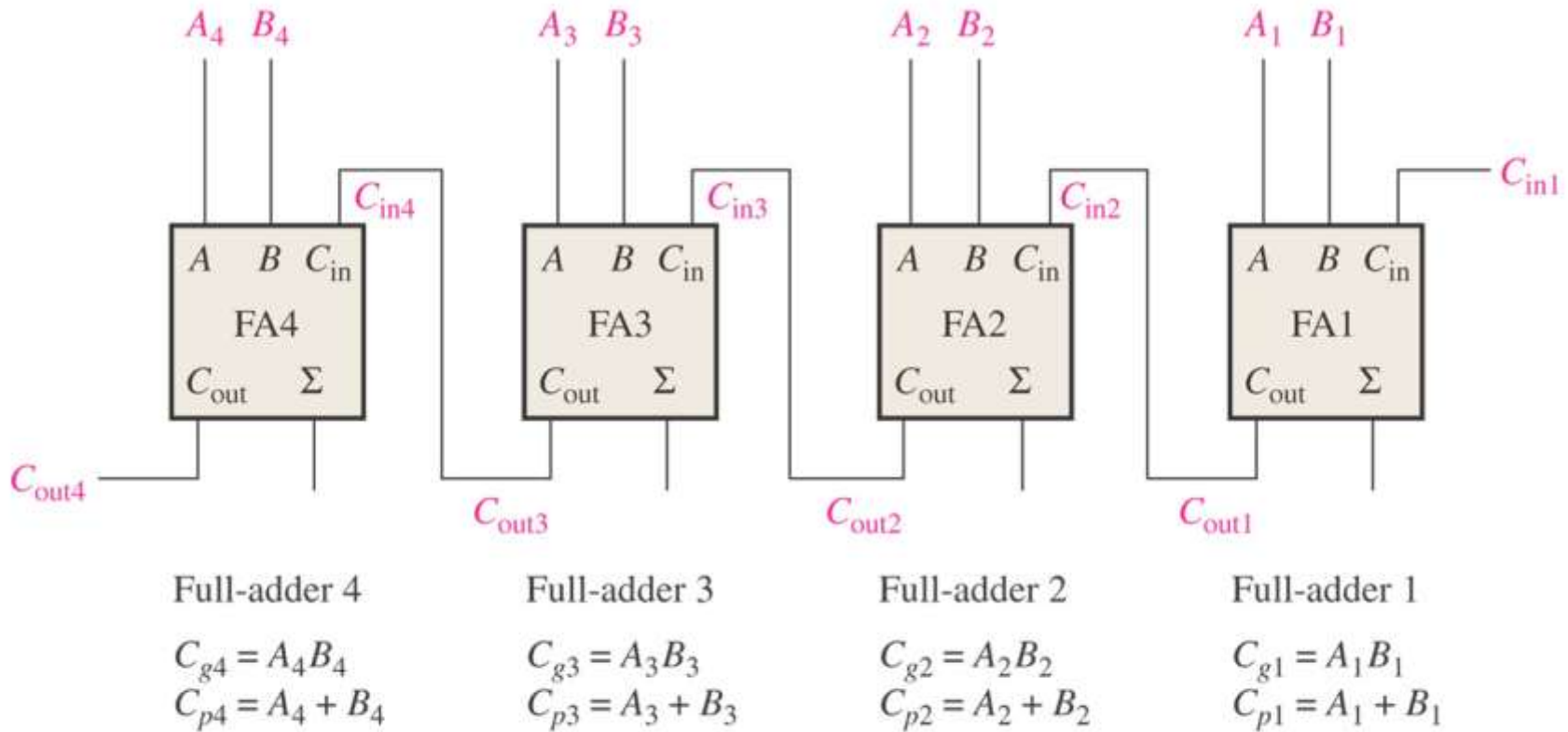
A	B	C _{in}	C _p	C _g	C _{out}
0	0	0	0	0	0
0	0	1	0	0	0
0	1	0	1	0	0
0	1	1	1	0	1
1	0	0	1	0	0
1	0	1	1	0	1
1	1	0	1	1	1
1	1	1	1	1	1

$$C_g = AB$$

$$C_p = A + B$$

$$C_{out} = C_g + C_p C_{in}$$

Figure 6–17 Carry generation and carry propagation in terms of the input bits to a 4-bit adder



$$C_{out1} = C_{g1} + C_{p1}C_{in1}$$

$$C_{in2} = C_{out1}$$

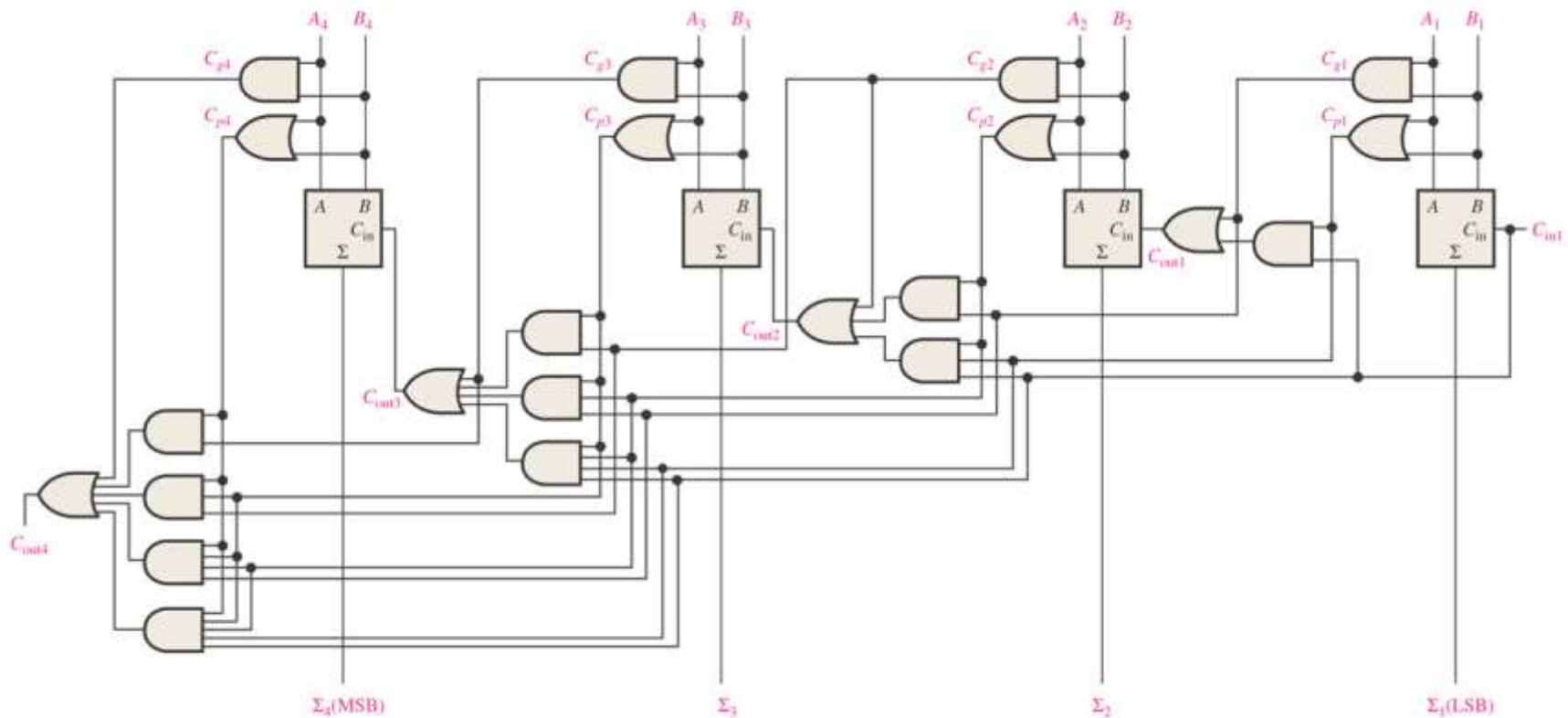
$$C_{out2} = C_{g2} + C_{p2}C_{in2} = C_{g2} + C_{p2}C_{out1} = C_{g2} + C_{p2}(C_{g1} + C_{p1}C_{in1})$$

$$C_{in3} = C_{out2}$$

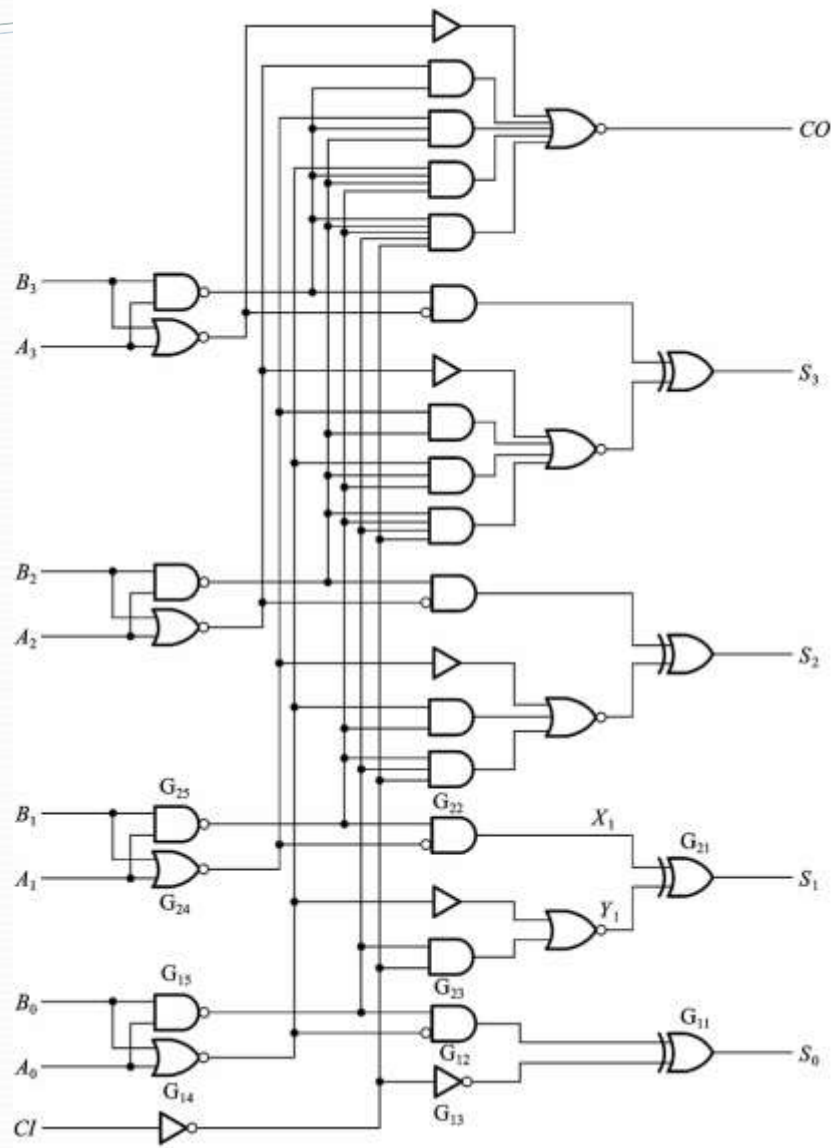
.....

$$C_{out3} = C_{g3} + C_{p3}C_{in3} = C_{g3} + C_{p3}C_{out2} = C_{g3} + C_{p3}(C_{g2} + C_{p2}(C_{g1} + C_{p1}C_{in1}))$$

Figure 6–18 Logic diagram for a 4-stage look-ahead carry adder.



The output carry for each full-adder stage is dependent only on the initial input carry (C_{in1}), the C_g and C_p functions of that stage, and the C_g and C_p functions of the preceding stages.



The logic diagram of a 4-stage look-ahead carry adder

6.4 Comparators

- Determine whether two numbers are equal
 - Equal
 - Inequal

Basic comparator operation.

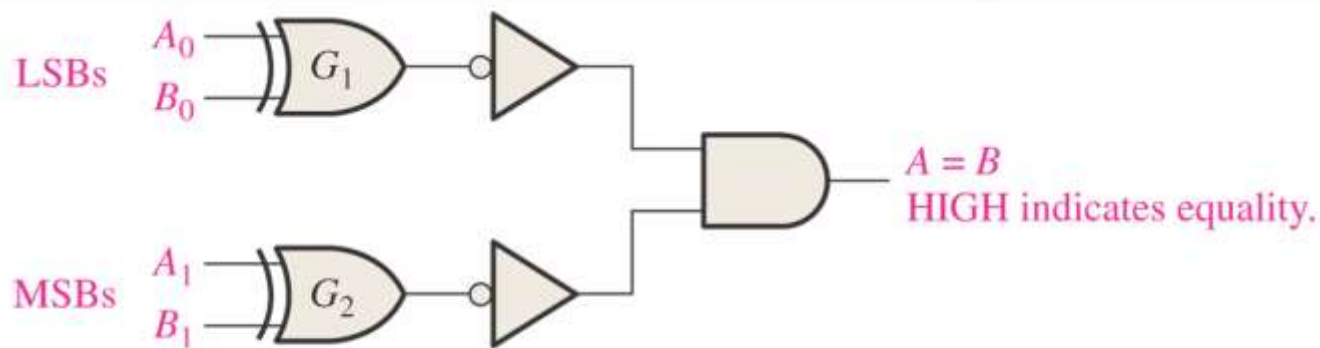
0 0
0 0 The input bits are equal.

1 0
0 0 The input bits are not equal.

0 1
1 1 The input bits are not equal.

1 1
1 1 The input bits are equal.

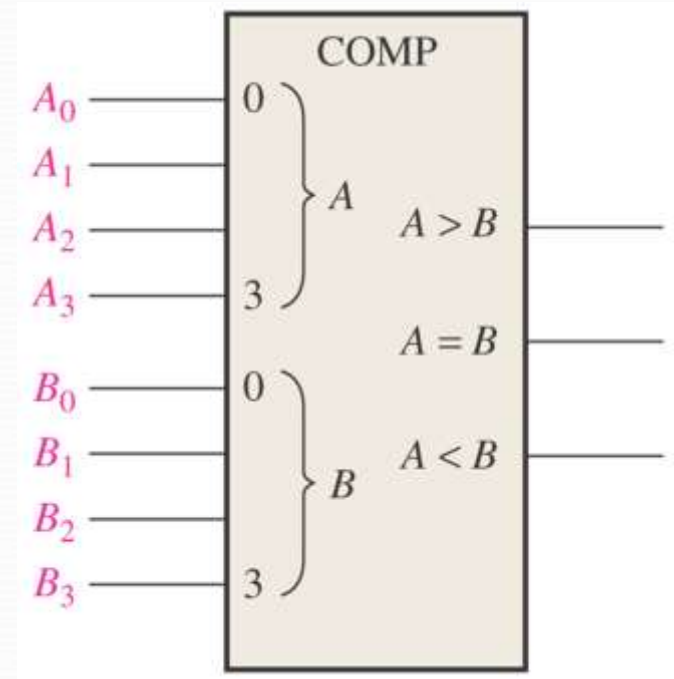
Logic diagram for equality comparison of two 2-bit numbers



General format: Binary number $A \rightarrow A_1A_0$
Binary number $B \rightarrow B_1B_0$

Logic symbol for a 4-bit comparator with inequality indication.

- 1, If $A_3=1$ and $B_3=0$, number A is greater than number B;
- 2, If $A_3=0$ and $B_3=1$, number A is less than number B;
- 3, If $A_3=B_3$, examine the next lower bit position for an inequality.



Compare $A_3A_2A_1A_0$ with $B_3B_2B_1B_0$

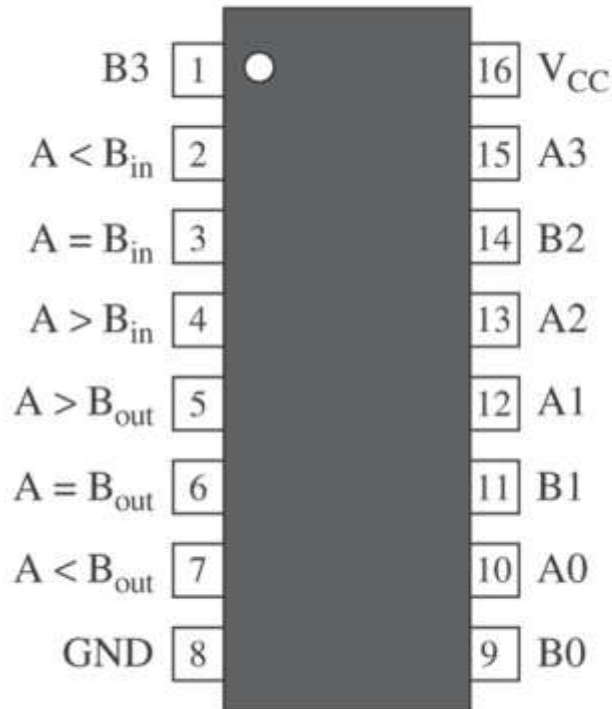
$$Y_{(A<B)} = \overline{A_3}B_3 + \overline{(A_3 \oplus B_3)}A_2B_2 + \overline{(A_3 \oplus B_3)(A_2 \oplus B_2)}A_1B_1 \\ + \overline{(A_3 \oplus B_3)(A_2 \oplus B_2)(A_1 \oplus B_1)}A_0B_0$$

$$Y_{(A=B)} = \overline{(A_3 \oplus B_3)(A_2 \oplus B_2)(A_1 \oplus B_1)(A_0 \oplus B_0)}$$

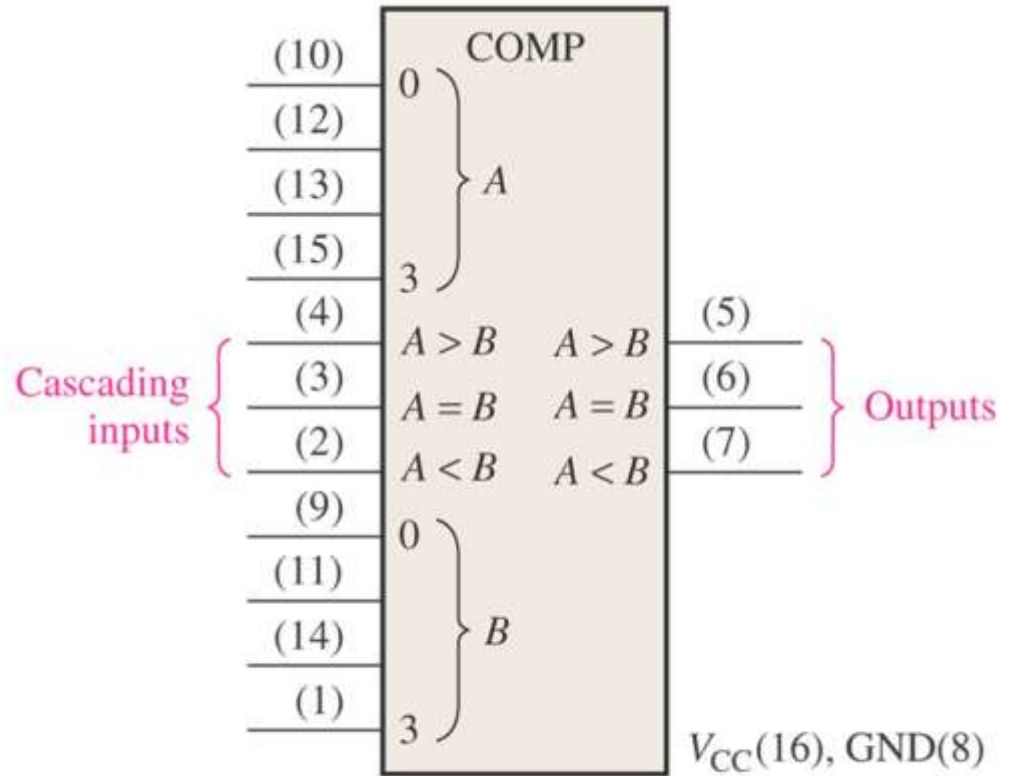
$$Y_{(A>B)} = \overline{(Y_{(A<B)} + Y_{(A=B)})}$$

**How to interpret the
above logic
expressions?**

Figure 6–24 Pin diagram and logic symbol for the 74HC85 4-bit magnitude comparator (pin numbers are in parentheses).

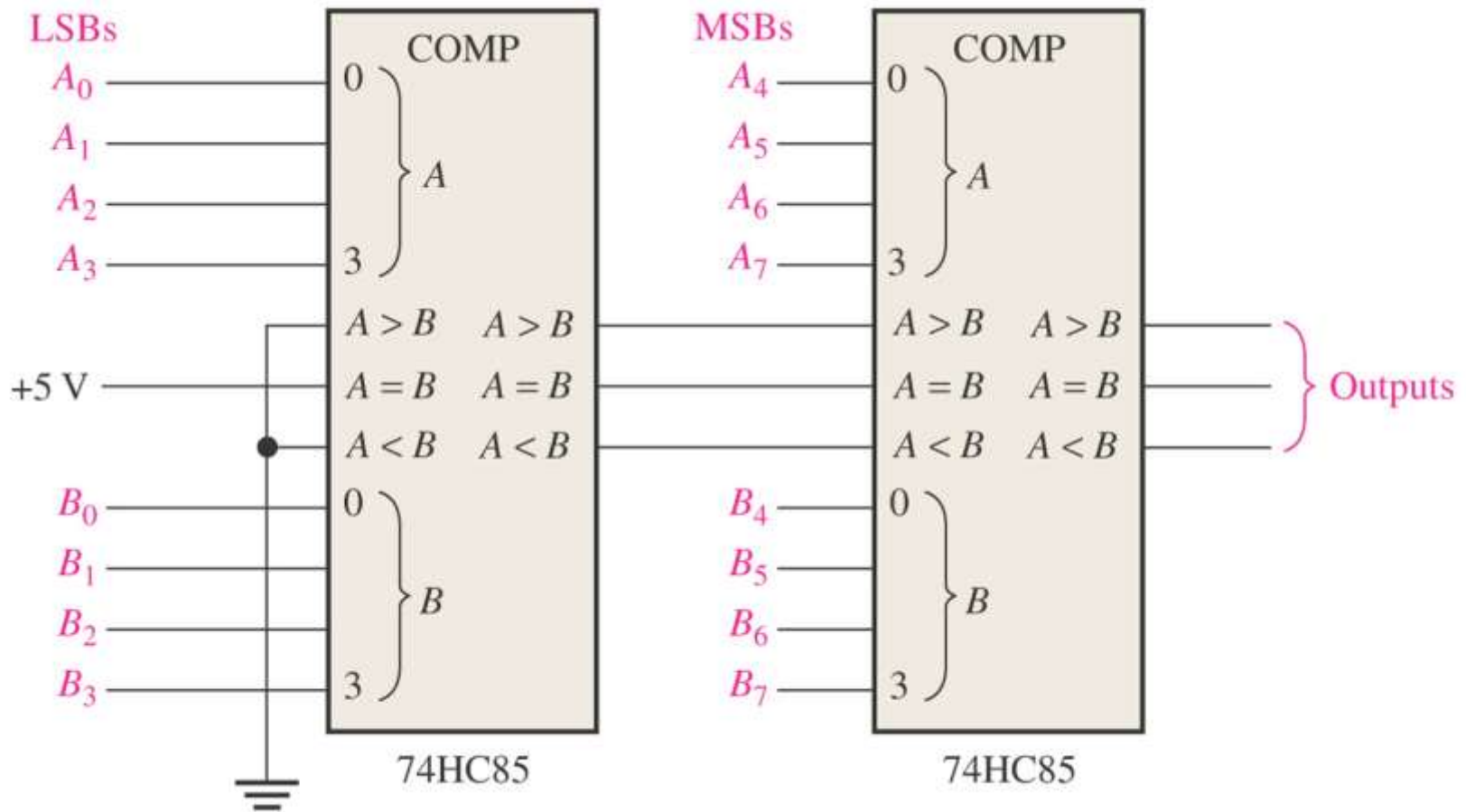


(a) Pin diagram



(b) Logic symbol

Example: An 8-bit magnitude comparator using two 74HC85s.



6.5 Decoders

- Decoder
 - Detects the presence of a specified combination of bits on its inputs
 - Indicates the presence of that code by a specified output level
 - Has n input lines and 2^n output lines

The basic binary decoder

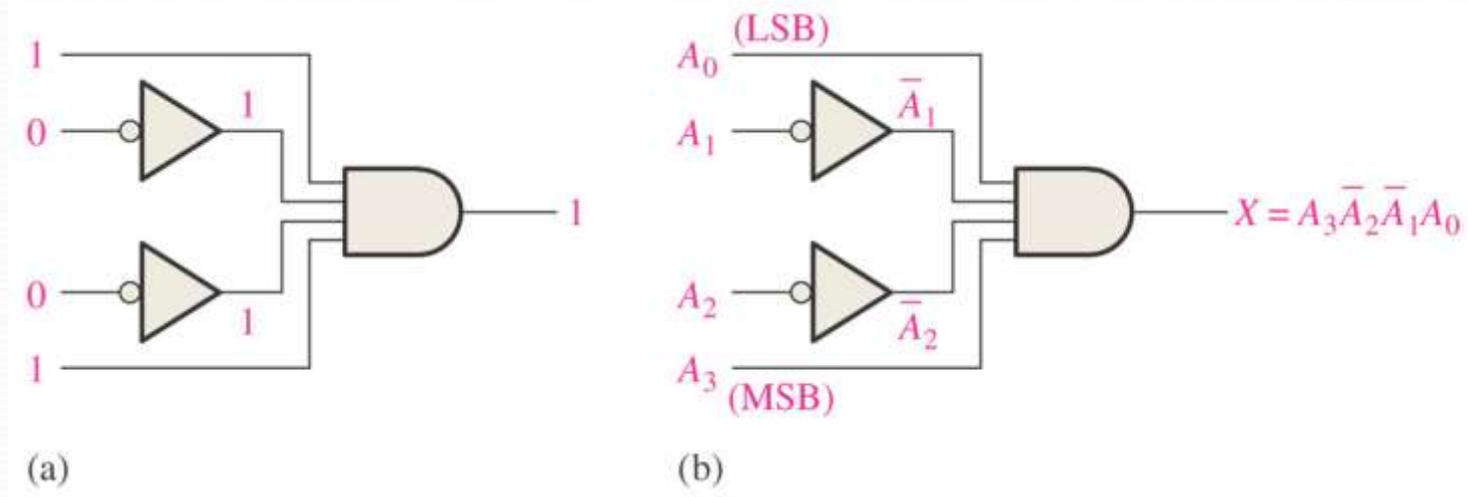


Figure 6–26 Decoding logic for the binary code 1001 with an active-HIGH output.

Example Decoding logic for producing a HIGH output when 1011 is on the inputs.

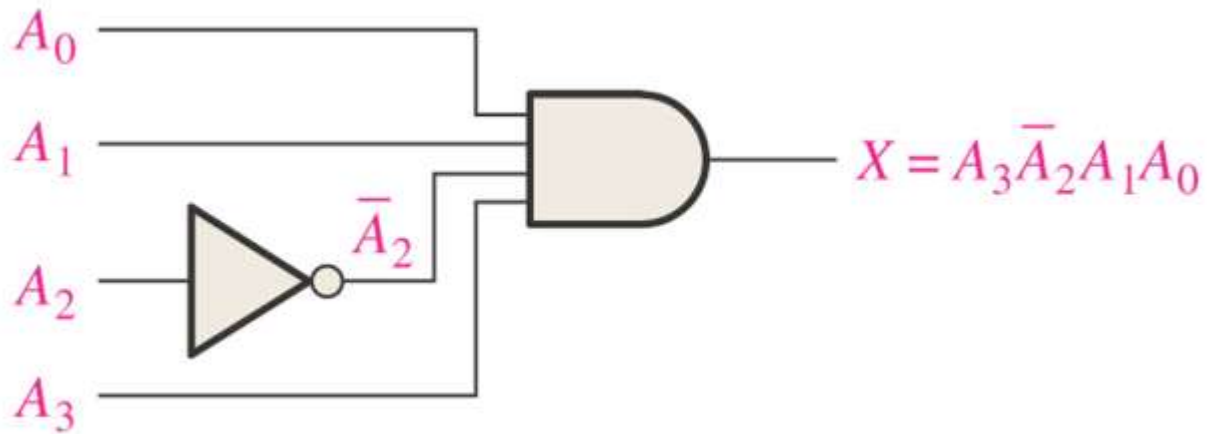
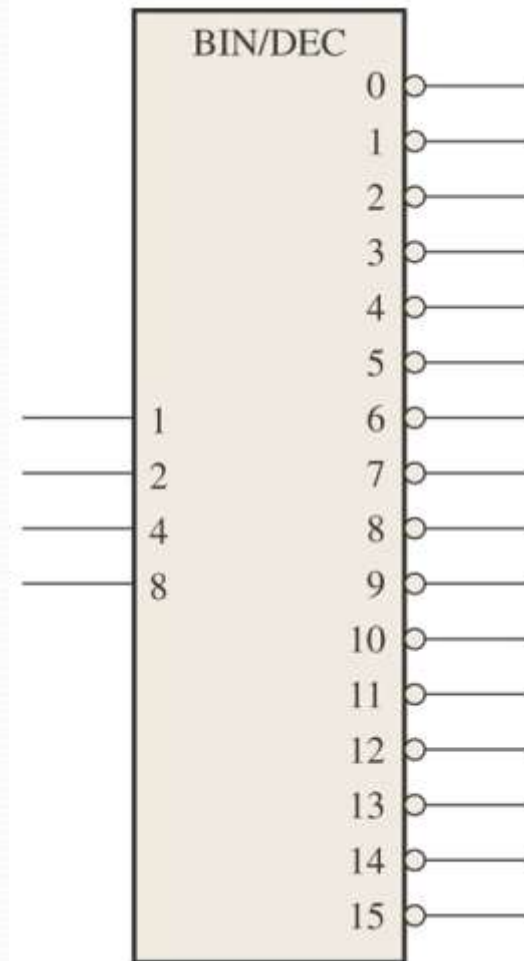


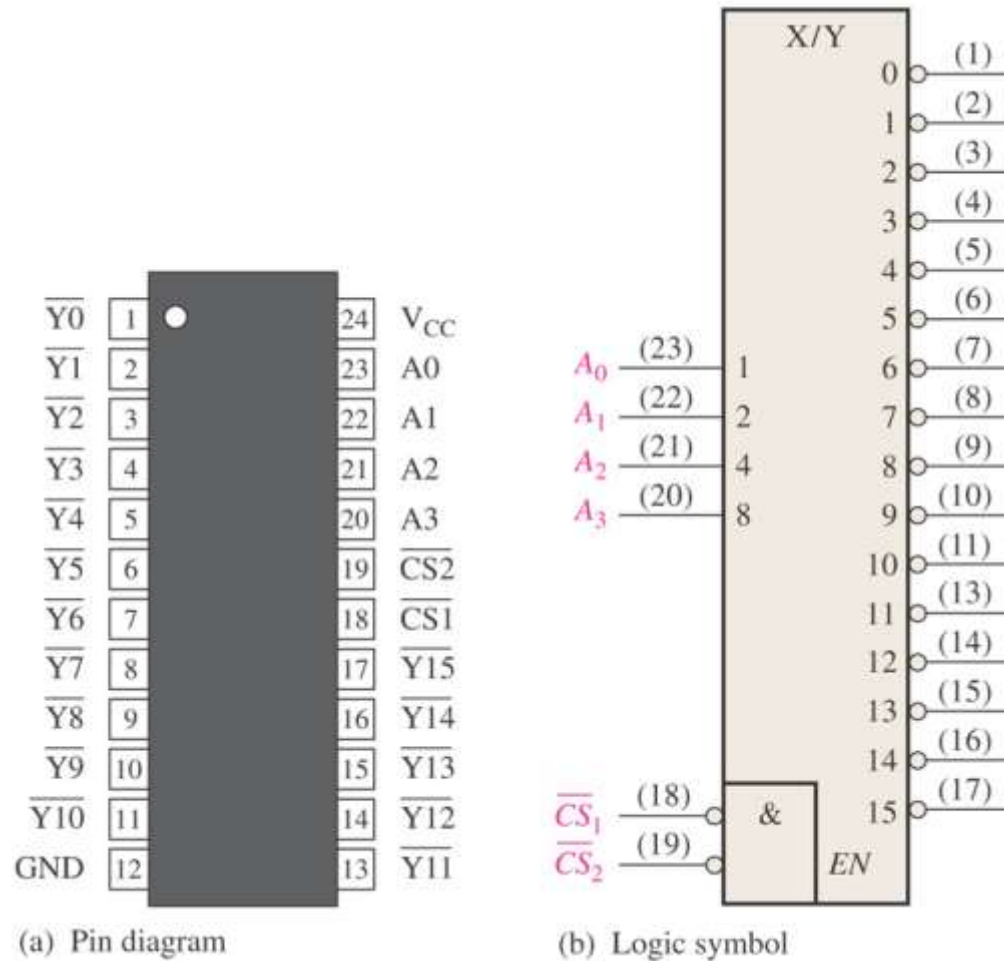
Figure 6–28 Logic symbol for a 4-line-to-16-line (1-of-16) decoder.



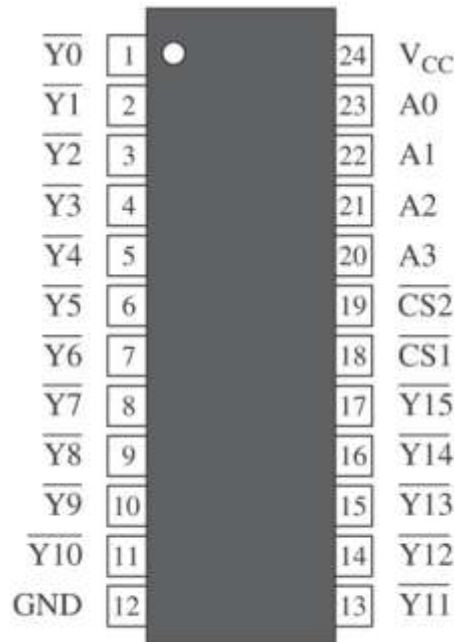
Truth table for a 4-16bit decoder

DECIMAL DIGIT	BINARY INPUTS				DECODING FUNCTION	OUTPUTS															
	A ₃	A ₂	A ₁	A ₀		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	0	0	0	$\bar{A}_3\bar{A}_2\bar{A}_1\bar{A}_0$	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	0	0	0	1	$\bar{A}_3\bar{A}_2\bar{A}_1A_0$	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2	0	0	1	0	$\bar{A}_3\bar{A}_2A_1\bar{A}_0$	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1
3	0	0	1	1	$\bar{A}_3\bar{A}_2A_1A_0$	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1
4	0	1	0	0	$\bar{A}_3A_2\bar{A}_1\bar{A}_0$	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1
5	0	1	0	1	$\bar{A}_3A_2\bar{A}_1A_0$	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1
6	0	1	1	0	$\bar{A}_3A_2A_1\bar{A}_0$	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1
7	0	1	1	1	$\bar{A}_3A_2A_1A_0$	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1
8	1	0	0	0	$A_3\bar{A}_2\bar{A}_1\bar{A}_0$	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1
9	1	0	0	1	$A_3\bar{A}_2\bar{A}_1A_0$	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1
10	1	0	1	0	$A_3\bar{A}_2A_1\bar{A}_0$	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1
11	1	0	1	1	$A_3\bar{A}_2A_1A_0$	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1
12	1	1	0	0	$A_3A_2\bar{A}_1\bar{A}_0$	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1
13	1	1	0	1	$A_3A_2\bar{A}_1A_0$	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1
14	1	1	1	0	$A_3A_2A_1\bar{A}_0$	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1
15	1	1	1	1	$A_3A_2A_1A_0$	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0

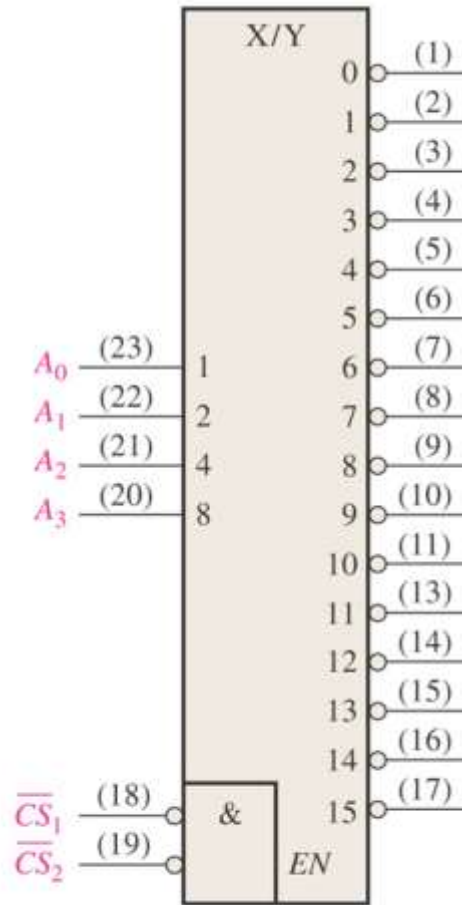
Figure 6–29 Pin diagram and logic symbol for the 74HC154 1-of-16 decoder.



Example: A certain application requires that a 5-bit number be decoded. Use 74HC154 decoders to implement the logic.

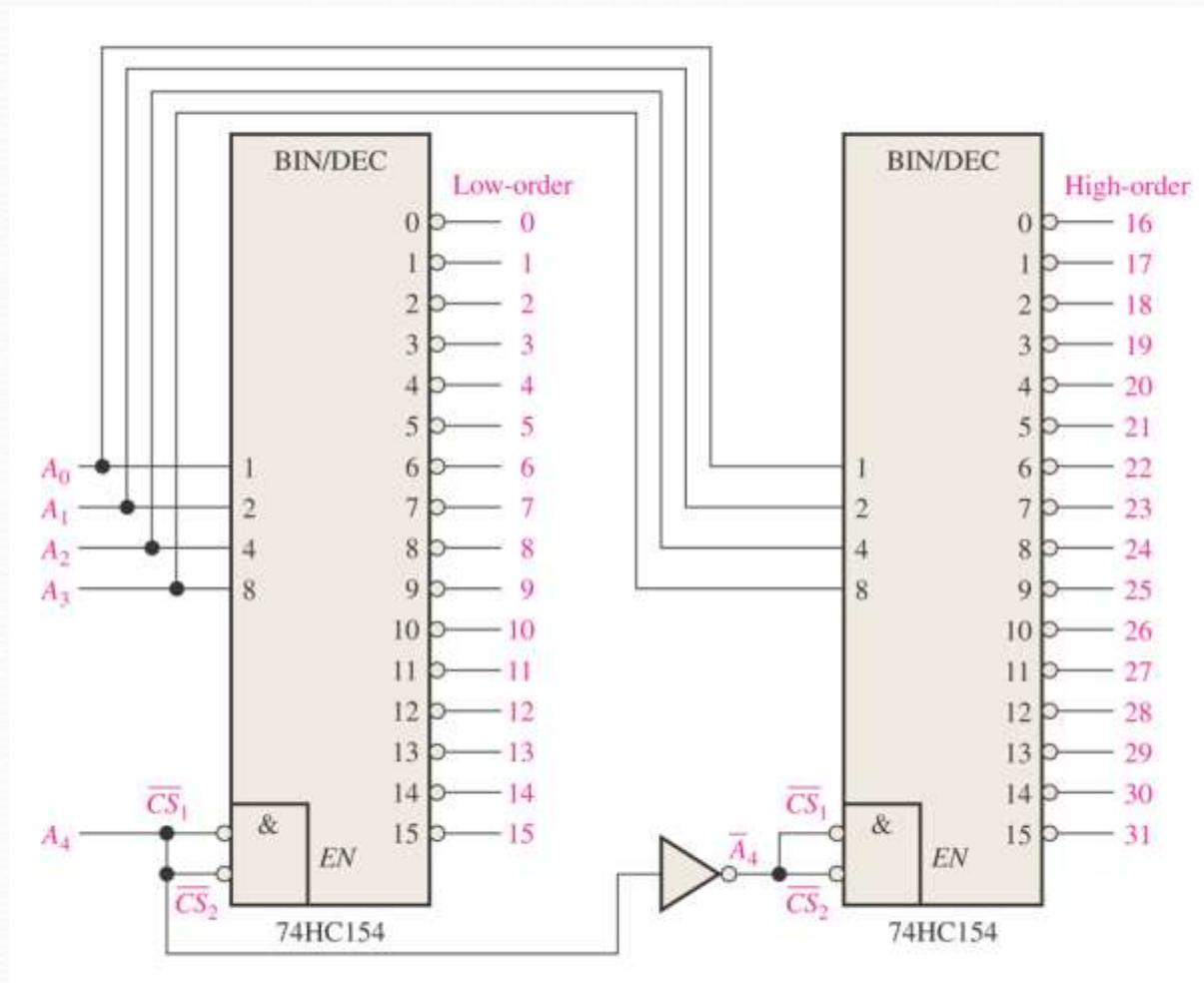


(a) Pin diagram

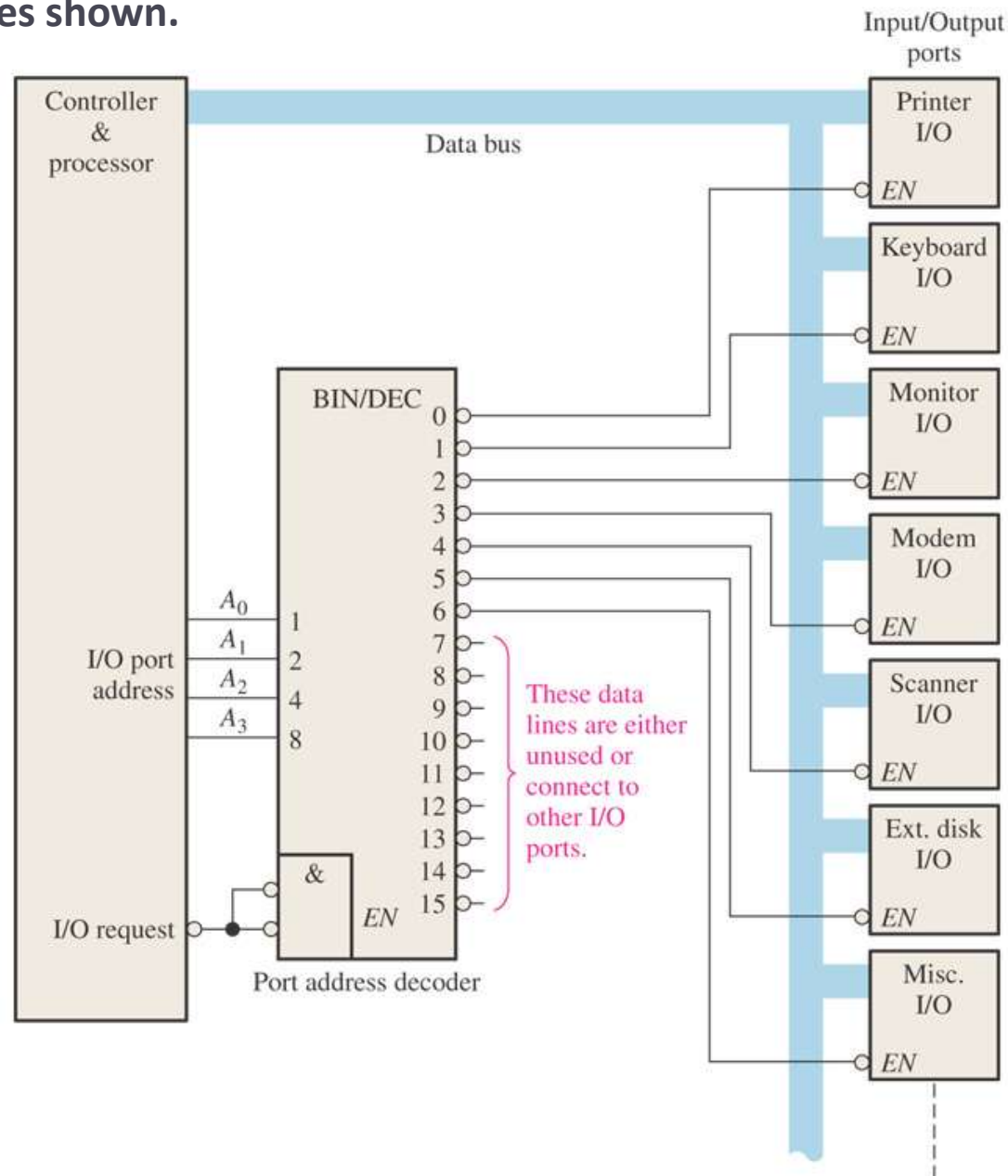


(b) Logic symbol

Example: A certain application requires that a 5-bit number be decoded. Use 74HC154 decoders to implement the logic.



An Application A simplified computer I/O port system with a port address decoder with only four address lines shown.

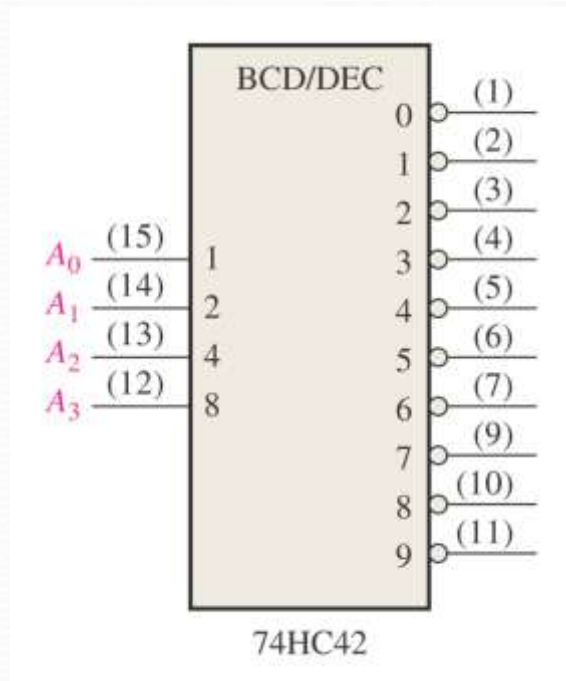


The BCD-to-Decimal Decoder

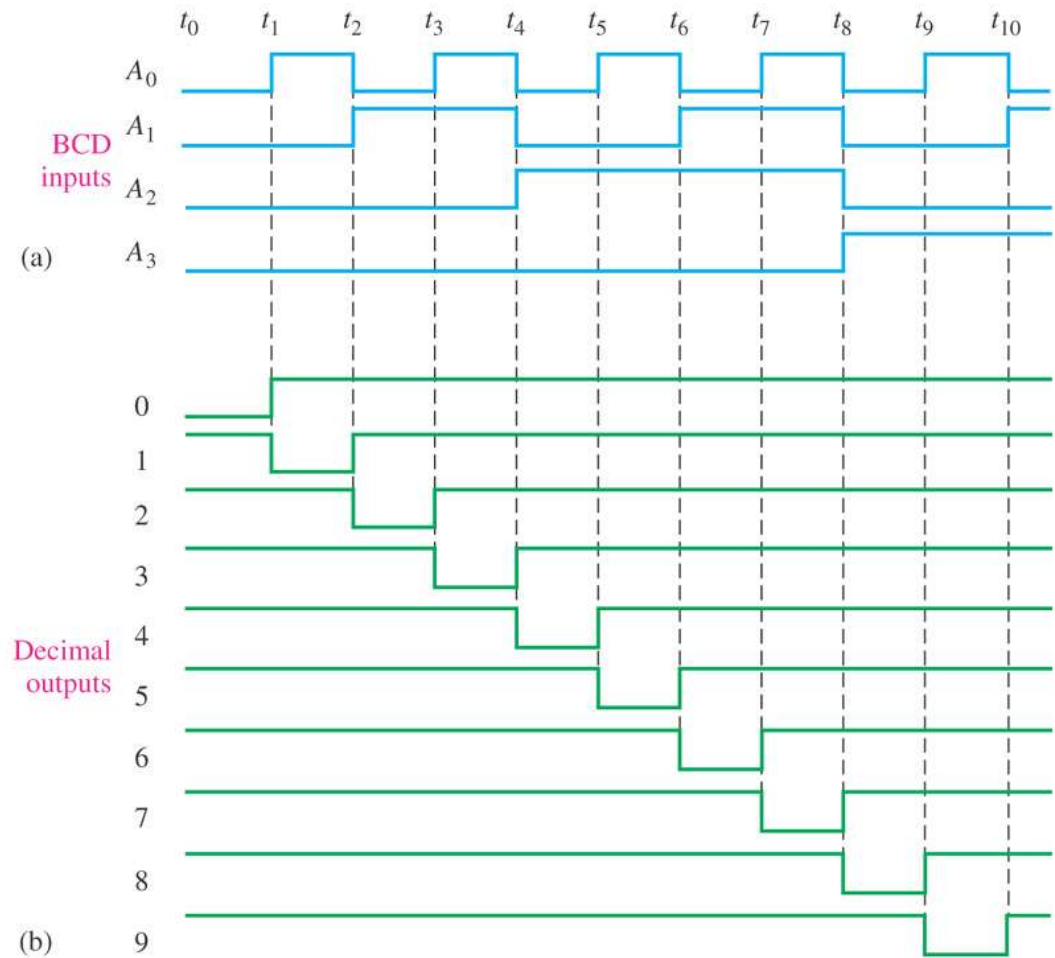
- Converts each BCD code (8421 code) into one of ten possible decimal digit indications

输 入				输 出									
A_3	A_2	A_1	A_0	Y_0	Y_1	Y_2	Y_3	Y_4	Y_5	Y_6	Y_7	Y_8	Y_9
0	0	0	0	0	1	1	1	1	1	1	1	1	1
0	0	0	1	1	0	1	1	1	1	1	1	1	1
0	0	1	0	1	1	0	1	1	1	1	1	1	1
0	0	1	1	1	1	1	0	1	1	1	1	1	1
0	1	0	0	1	1	1	1	0	1	1	1	1	1
0	1	0	1	1	1	1	1	1	0	1	1	1	1
0	1	1	0	1	1	1	1	1	1	0	1	1	1
0	1	1	1	1	1	1	1	1	1	1	0	1	1
1	0	0	0	1	1	1	1	1	1	1	1	0	1
1	0	0	1	1	1	1	1	1	1	1	1	1	0
1	0	1	0	1	1	1	1	1	1	1	1	1	1
1	0	1	1	1	1	1	1	1	1	1	1	1	1
1	1	0	0	1	1	1	1	1	1	1	1	1	1
1	1	0	1	1	1	1	1	1	1	1	1	1	1
1	1	1	0	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1

Figure 6–32 The 74HC42 BCD-to-decimal



Active-LOW outputs



The BCD-to-7-Segment Decoder

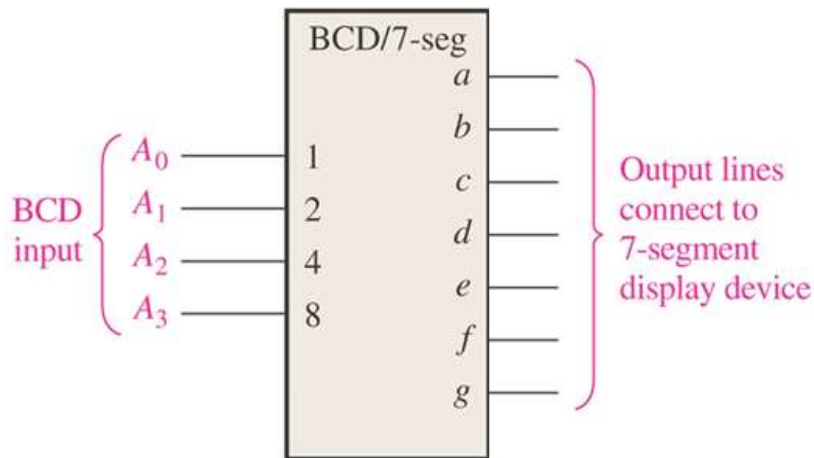
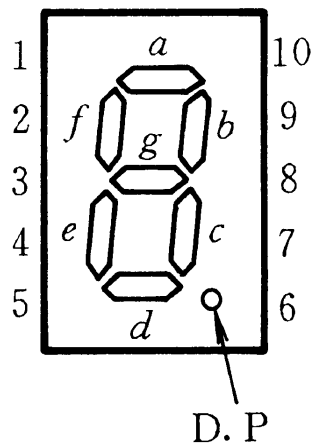
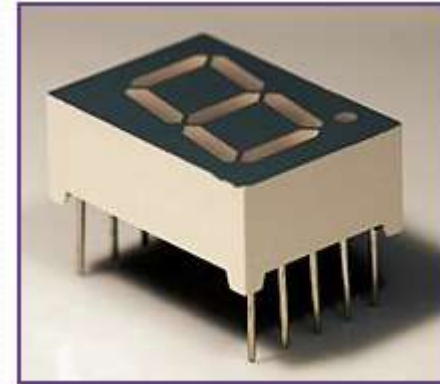
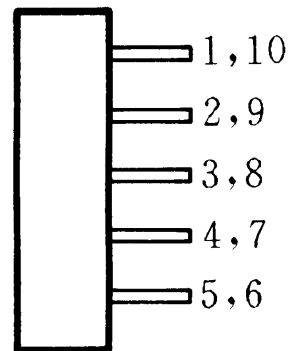


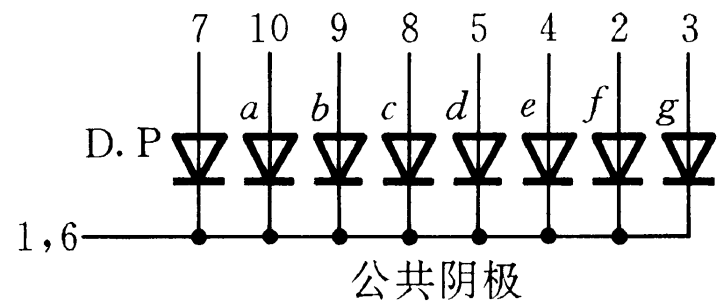
Figure 6-34 Logic symbol for a BCD-to-7-segment decoder/driver with active-High outputs.



(a)

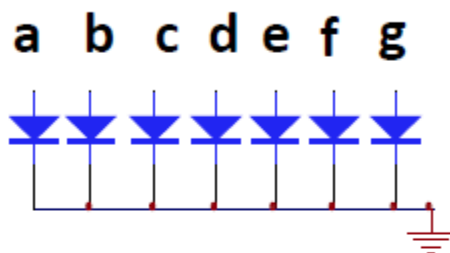


7(8)-segment display device



(b)

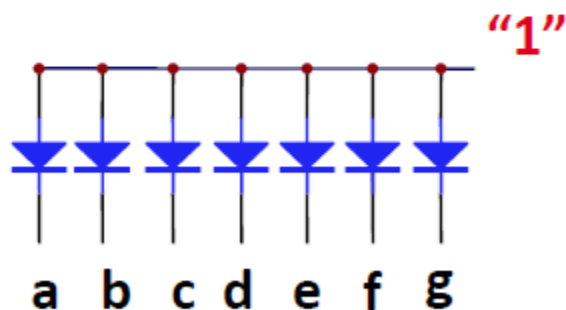
Connection: 连接方式不同分成共阴极和共阳极两种



BS201A

Common-cathode (共阴极)

diodes $\longrightarrow$ logic high $\longrightarrow$ bright



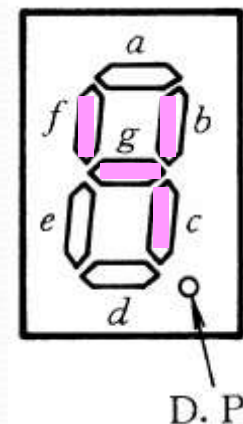
BS201B

Common-anode (共阳极)

diodes $\longrightarrow$ logic low $\longrightarrow$ bright

The BCD-to-7-Segment Decoder

输 入					输 出							字形
数字	A_3	A_2	A_1	A_0	Y_a	Y_b	Y_c	Y_d	Y_e	Y_f	Y_g	
0	0	0	0	0	1	1	1	1	1	1	0	
1	0	0	0	1	0	1	1	0	0	0	0	
2	0	0	1	0	1	1	0	1	1	0	1	
3	0	0	1	1	1	1	1	1	0	0	1	
4	0	1	0	0	0	1	1	0	0	1	1	
5	0	1	0	1	1	0	1	1	0	1	1	
6	0	1	1	0	0	0	1	1	1	1	1	
7	0	1	1	1	1	1	1	0	0	0	0	
8	1	0	0	0	1	1	1	1	1	1	1	
9	1	0	0	1	1	1	1	0	0	1	1	
10	1	0	1	0	0	0	0	1	1	0	1	
11	1	0	1	1	0	0	1	1	0	0	1	
12	1	1	0	0	0	1	0	0	0	1	1	
13	1	1	0	1	1	0	0	1	0	1	1	
14	1	1	1	0	0	0	0	1	1	1	1	
15	1	1	1	1	0	0	0	0	0	0	0	



Truth table Karnaugh Map

$A_1 A_2$

$A_3 A_2$

	00	01	11	10
00	1	0	1	1
01	0	1	1	0
11	0	1	0	0
10	1	1	0	0

(a)

$A_1 A_2$

$A_3 A_2$

	00	01	11	10
00	1	1	1	1
01	1	0	1	0
11	1	0	0	0
10	1	1	0	0

(b)

$A_1 A_2$

$A_3 A_2$

	00	01	11	10
00	1	1	1	0
01	1	1	1	1
11	0	0	0	0
10	1	1	1	0

(c)

$A_1 A_2$

$A_3 A_2$

	00	01	11	10
00	1	0	1	1
01	0	1	0	1
11	0	1	0	1
10	1	0	1	1

(d)

$A_1 A_2$

$A_3 A_2$

	00	01	11	10
00	1	0	0	1
01	0	0	0	1
11	0	0	0	1
10	1	0	0	1

(e)

$A_1 A_2$

$A_3 A_2$

	00	01	11	10
00	1	0	0	0
01	1	1	0	1
11	1	1	0	1
10	1	1	0	0

(f)

$A_1 A_2$

$A_3 A_2$

	00	01	11	10
00	0	0	1	1
01	1	1	0	1
11	1	1	0	1
10	1	1	1	1

(g)

The logic diagram of BCD to 7-segment display device (7447)

$$Y_a = \overline{(\overline{A_3 A_2 A_1 A_0} + A_3 A_1 + A_2 \overline{A_0})}$$

$$Y_b = \overline{(A_3 A_1 + A_2 A_1 \overline{A_0} + A_2 A_1 A_0)}$$

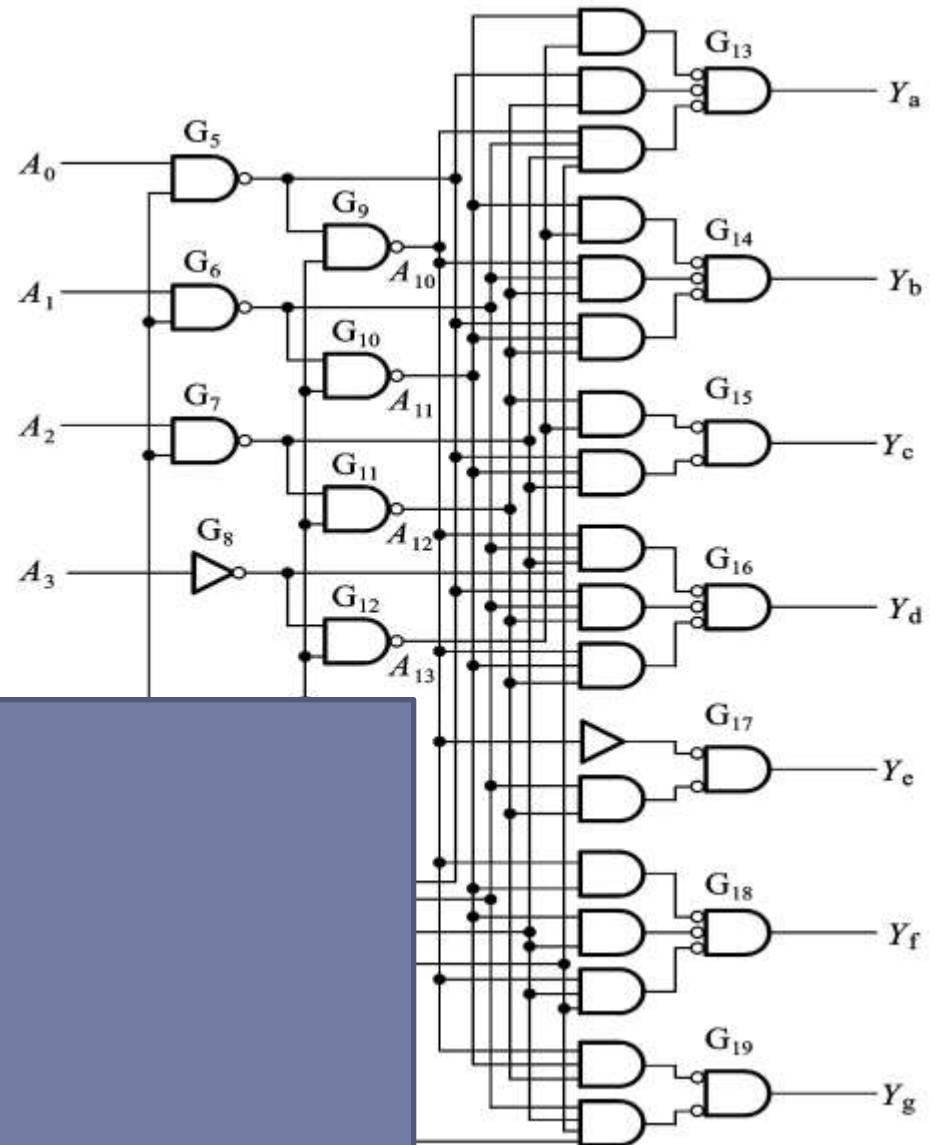
$$Y_c = \overline{(A_3 A_2 + \overline{A_2 A_1 A_0})}$$

$$Y_d = \overline{(A_2 A_1 A_0 + \overline{A_2 A_1 A_0} + \overline{A_2 A_1 A_0})}$$

$$Y_e = \overline{(A_2 \overline{A_1} + A_0)}$$

$$Y_f = \overline{(\overline{A_3 A_2 A_0} + \overline{A_2 A_1} + A_1 A_0)}$$

$$Y_g = \overline{(\overline{A_3 A_2 A_1} + A_2 A_1 A_0)}$$



The logic diagram of BCD to 7-segment display device (7447)

$$Y_a = \overline{(A_3 A_2 A_1 A_0 + A_3 A_1 + A_2 A_0)}$$

$$Y_b = \overline{(A_3 A_1 + A_2 A_1 A_0 + A_2 A_1 A_0)}$$

$$Y_c = \overline{(A_3 A_2 + A_2 A_1 A_0)}$$

$$Y_d = \overline{(A_2 A_1 A_0 + A_2 A_1 A_0 + A_2 A_1 A_0)}$$

$$Y_e = \overline{(A_2 A_1 + A_0)}$$

$$Y_f = \overline{(A_3 A_2 A_0 + A_2 A_1 + A_1 A_0)}$$

$$Y_g = \overline{(A_3 A_2 A_1 + A_2 A_1 A_0)}$$

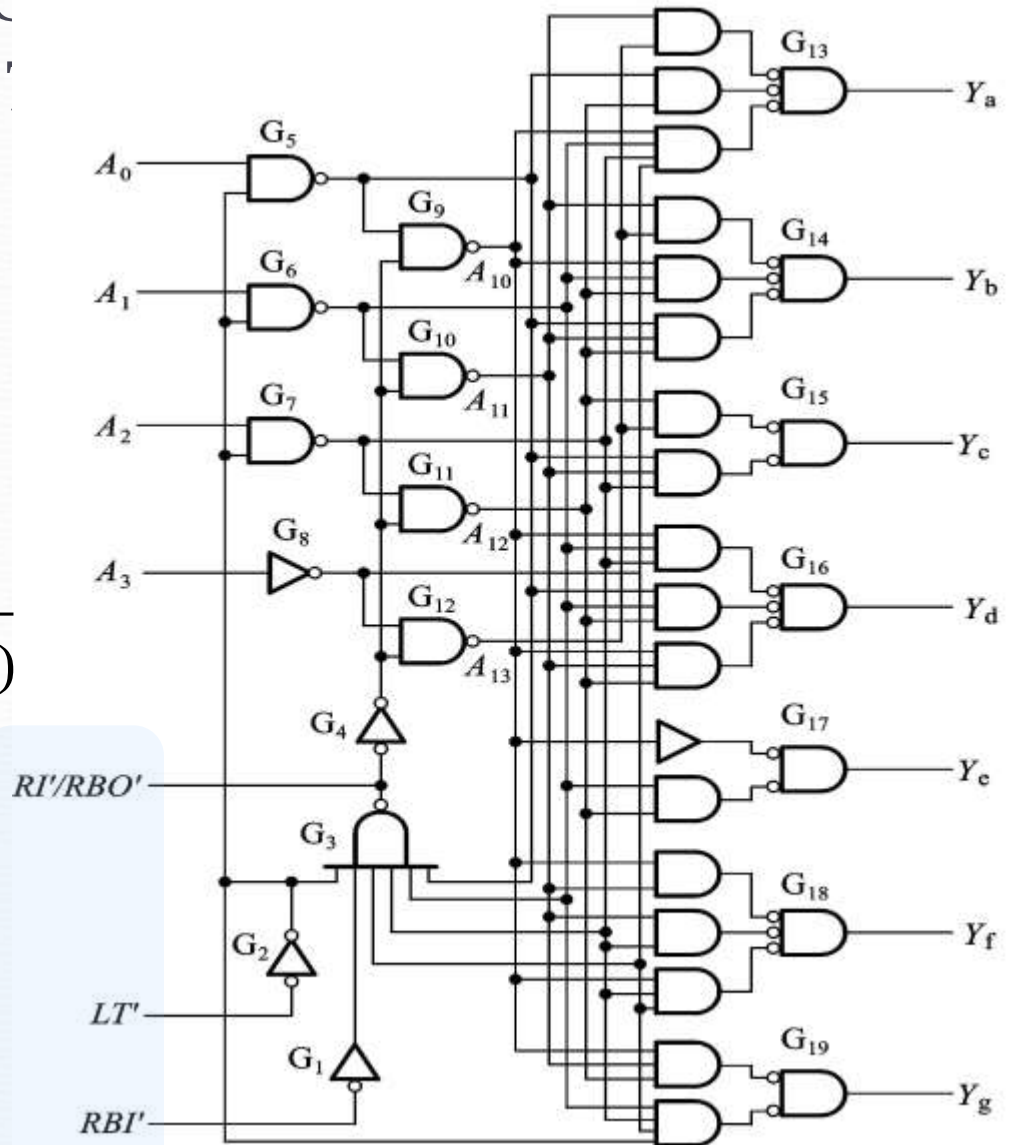
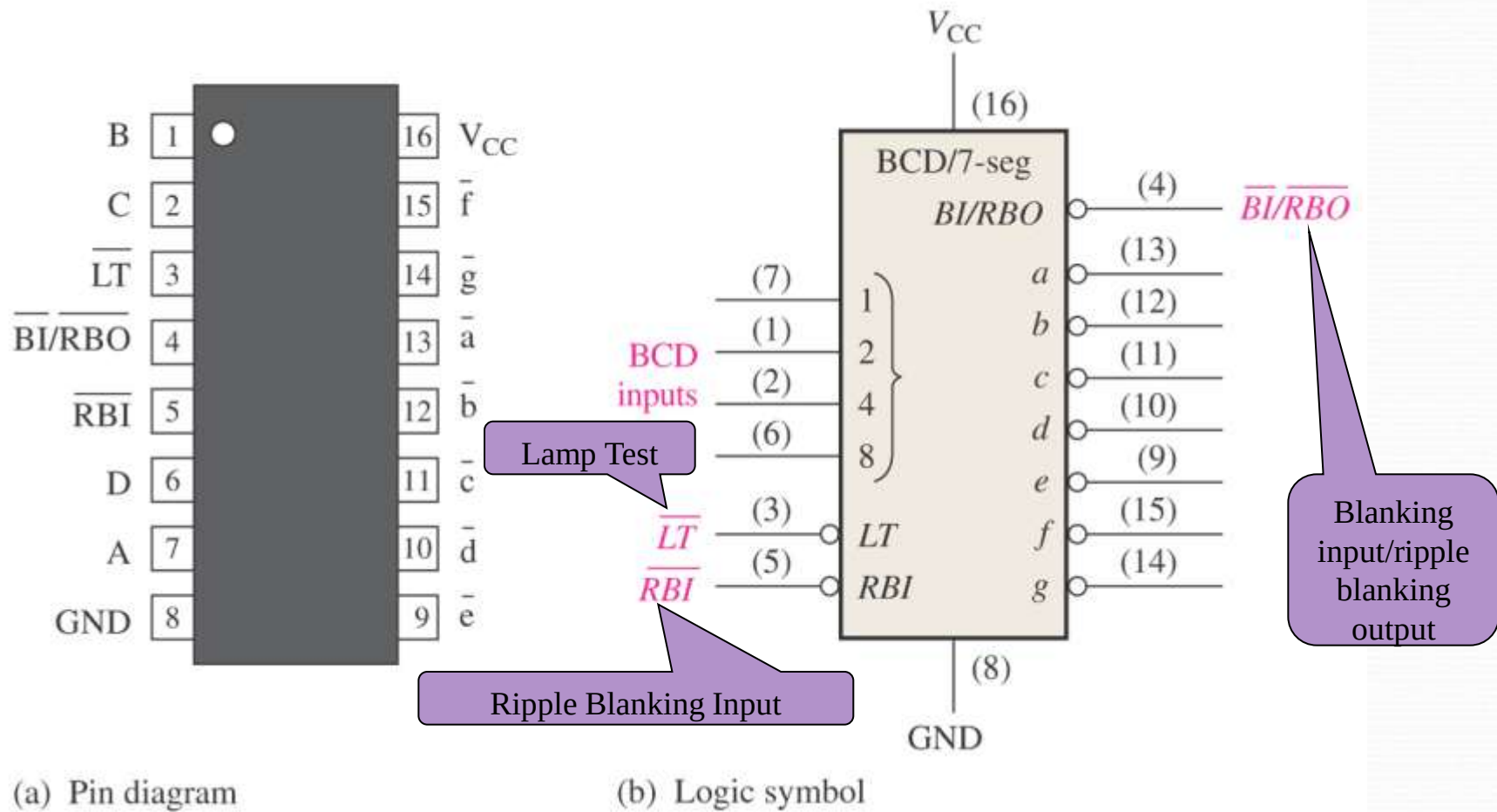


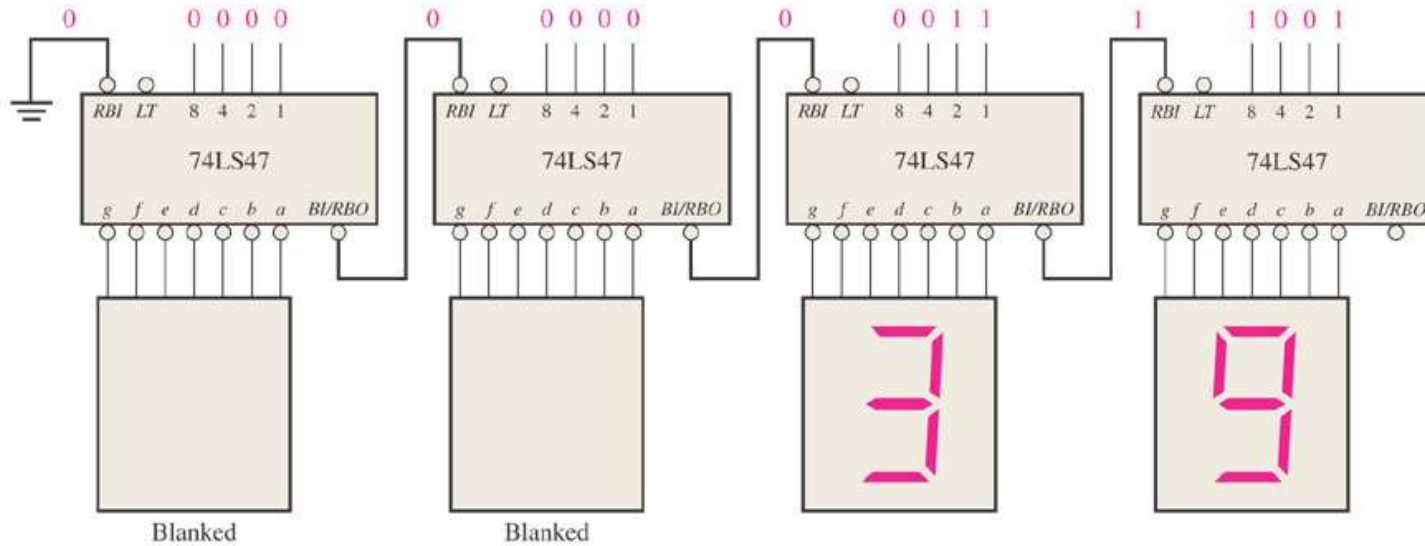
Figure 6–35 Pin diagram and logic symbol for the 74LS47 BCD-to-7-segment decoder/driver.



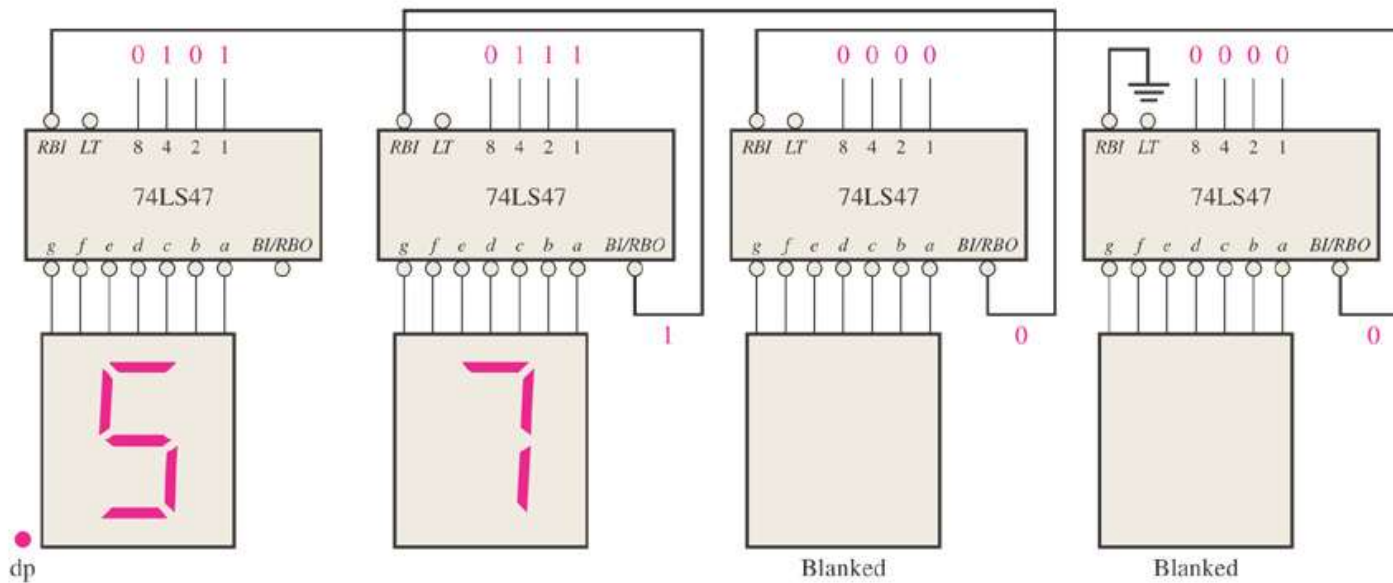
七段显示译码器 7448 的逻辑功能表

功能 (输入)	输入		输入/输出		输出							显示 字形
	$\overline{LT}$	$\overline{RBI}$	$A_3 A_2 A_1 A_0$	$\overline{BI/RBO}$	a	b	c	d	e	f	g	
0	1	1	0 0 0 0	1	1	1	1	1	1	1	0	□
1	1	×	0 0 0 1	1	0	1	1	0	0	0	0	1
2	1	×	0 0 1 0	1	1	1	0	1	1	0	1	2
3	1	×	0 0 1 1	1	1	1	1	1	0	0	1	3
4	1	×	0 1 0 0	1	0	1	1	0	0	1	1	4
5	1	×	0 1 0 1	1	1	0	1	1	0	1	1	5
6	1	×	0 1 1 0	1	0	0	1	1	1	1	1	6
7	1	×	0 1 1 1	1	1	1	1	0	0	0	0	7
8	1	×	1 0 0 0	1	1	1	1	1	1	1	1	8
9	1	×	1 0 0 1	1	1	1	1	0	0	1	1	9
10	1	×	1 0 1 0	1	0	0	0	1	1	0	1	□
11	1	×	1 0 1 1	1	0	0	1	1	0	0	1	□
12	1	×	1 1 0 0	1	0	1	0	0	0	1	1	□
13	1	×	1 1 0 1	1	1	0	0	1	0	1	1	□
14	1	×	1 1 1 0	1	0	0	0	1	1	1	1	□
15	1	×	1 1 1 1	1	0	0	0	0	0	0	0	
灭灯	×	×	×	×	×	×	×	×	×	×	×	
灭零	1	0	0 0 0 0	0	0	0	0	0	0	0	0	
试灯	0	×	×	×	×	×	×	×	×	×	×	□

Figure 6–36 Examples of zero suppression using the 74LS47 BCD to 7-segment decoder/driver.



(a) Illustration of leading zero suppression



(b) Illustration of trailing zero suppression

Encoders

- Performs a 'reverse' decoder function
- Accepts an active level on one of its inputs representing a digit and converts it to a coded output

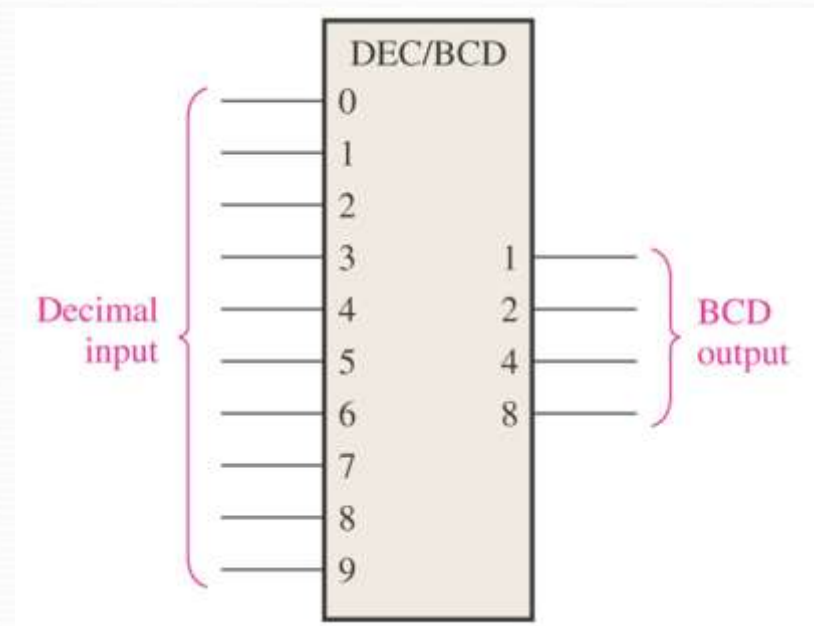


Figure 6–37 Logic symbol for a decimal-to-BCD encoder.

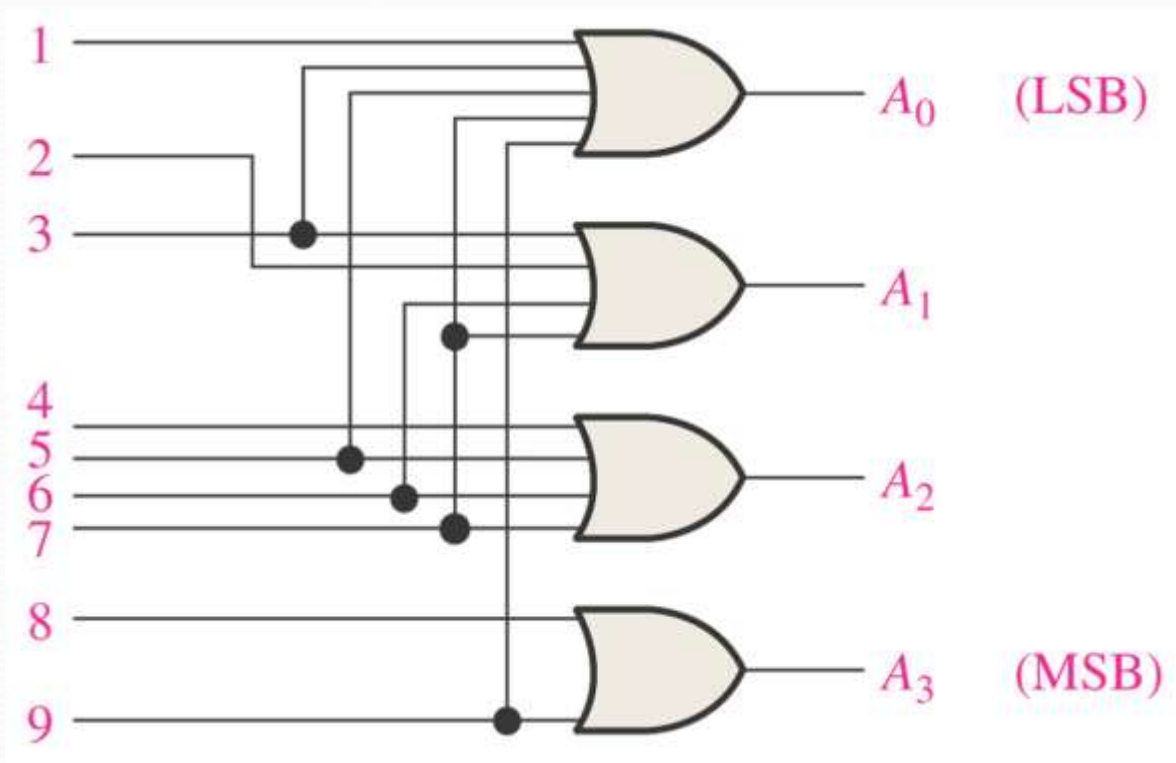


Figure 6–38 Basic logic diagram of a decimal-to-BCD encoder. A 0-digit input is not needed because the BCD outputs are all LOW when there are no HIGH inputs.

Figure 6–39 Pin diagram and logic symbol for the 74HC147 decimal-to-BCD **priority** encoder (HPRI means highest value input has priority). （优先编码器）

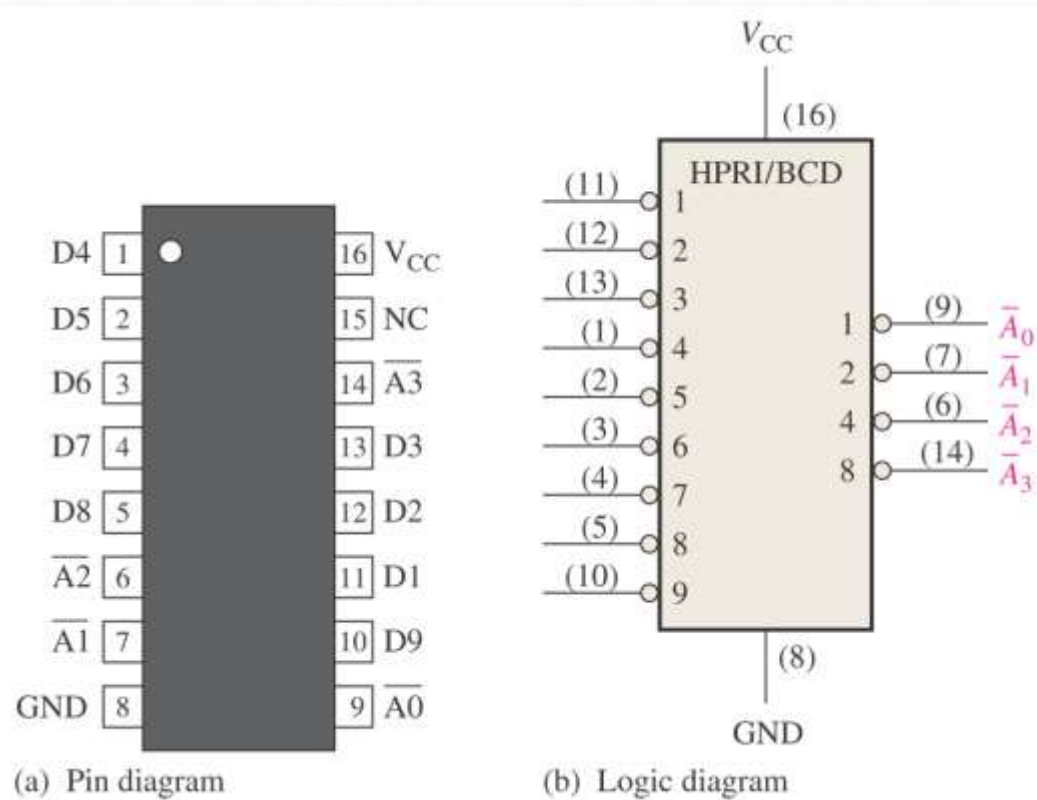
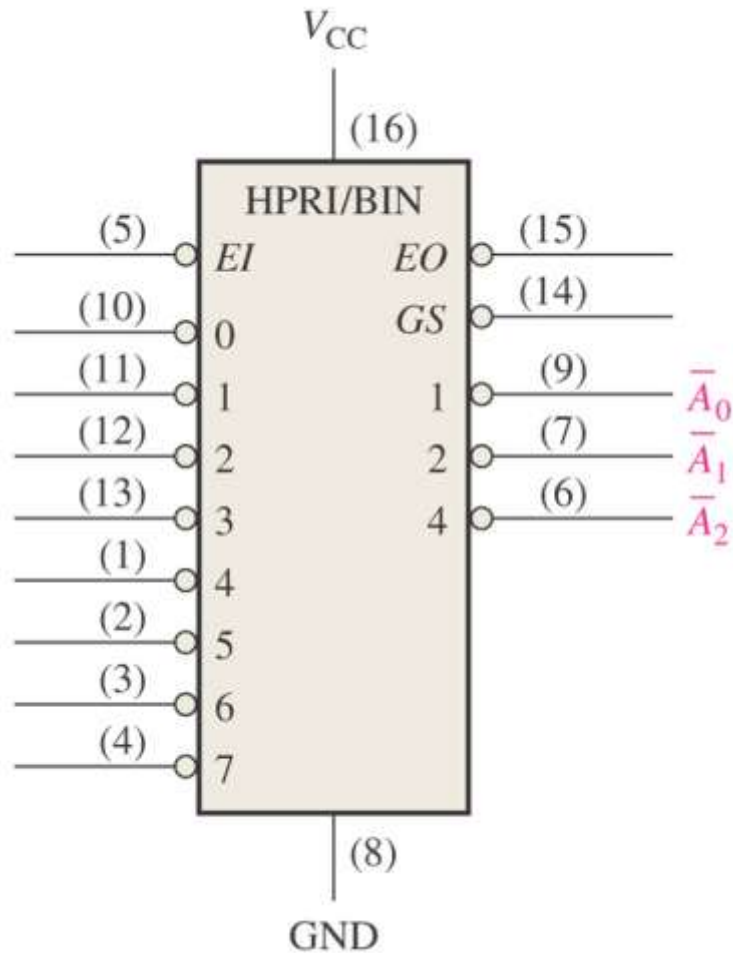


Figure 6–40 Logic symbol for the 74LS148 8-line-to-3-line encoder.



EI Enable input (active LOW)

EO Enable output (LOW when EI is LOW and none active inputs)

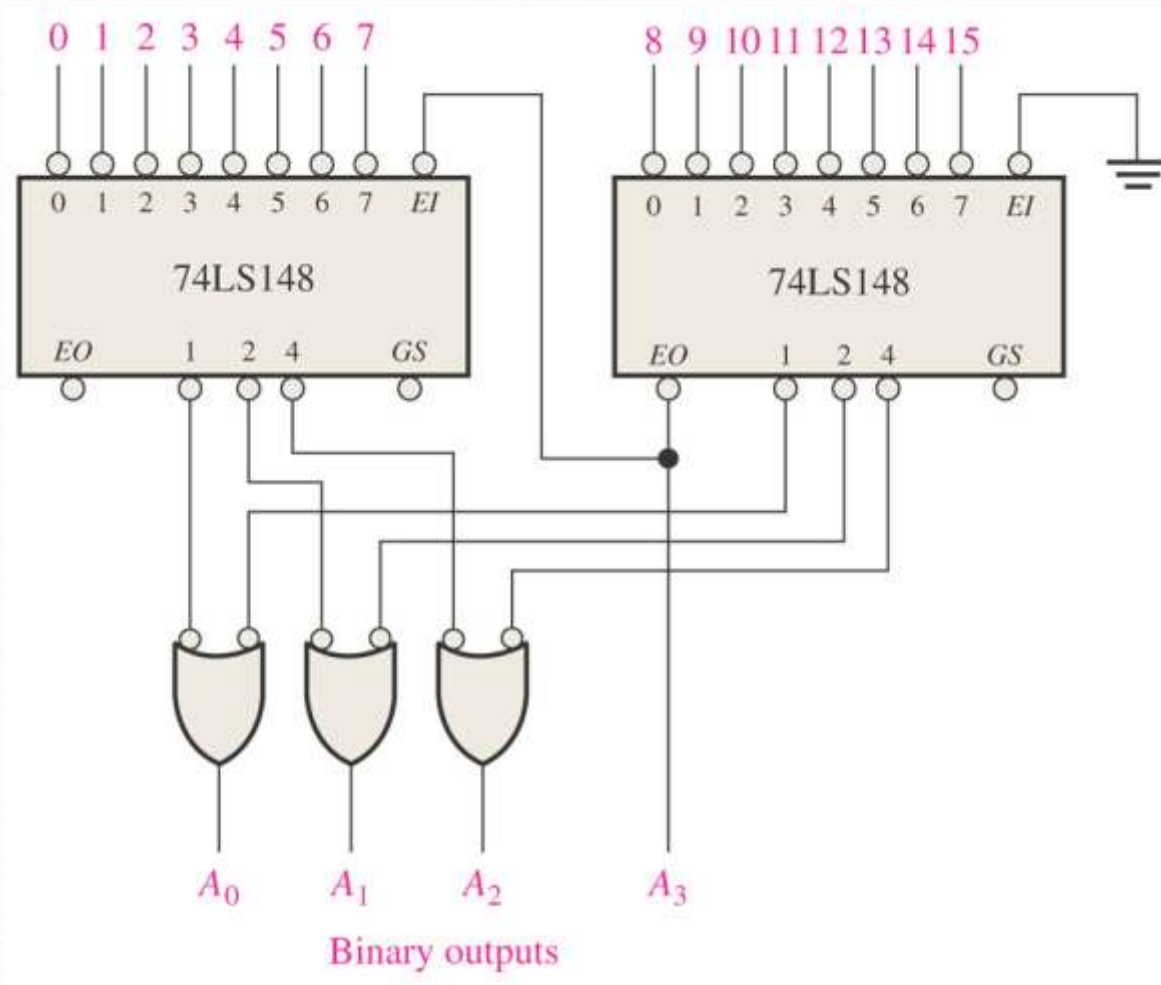
GS Output for expansion (LOW when EI is LOW and any of the inputs is active)

Truth table of LS74LS148

Inputs									Outputs				
$\overline{EI}$	$\overline{I_0}$	$\overline{I_1}$	$\overline{I_2}$	$\overline{I_3}$	$\overline{I_4}$	$\overline{I_5}$	$\overline{I_6}$	$\overline{I_7}$	$\overline{Y_2}$	$\overline{Y_1}$	$\overline{Y_0}$	$\overline{EO}$	$\overline{GS}$
1	X	X	X	X	X	X	X	X	1	1	1	1	1
0	1	1	1	1	1	1	1	1	1	1	1	0	1
0	X	X	X	X	X	X	X	0	0	0	0	1	0
0	X	X	X	X	X	X	0	1	0	0	1	1	0
0	X	X	X	X	X	0	1	1	0	1	0	1	0
0	X	X	X	X	0	1	1	1	0	1	1	1	0
0	X	X	X	0	1	1	1	1	1	0	0	1	0
0	X	X	0	1	1	1	1	1	1	0	1	1	0
0	X	0	1	1	1	1	1	1	1	1	0	1	0
0	0	1	1	1	1	1	1	1	1	1	1	1	0

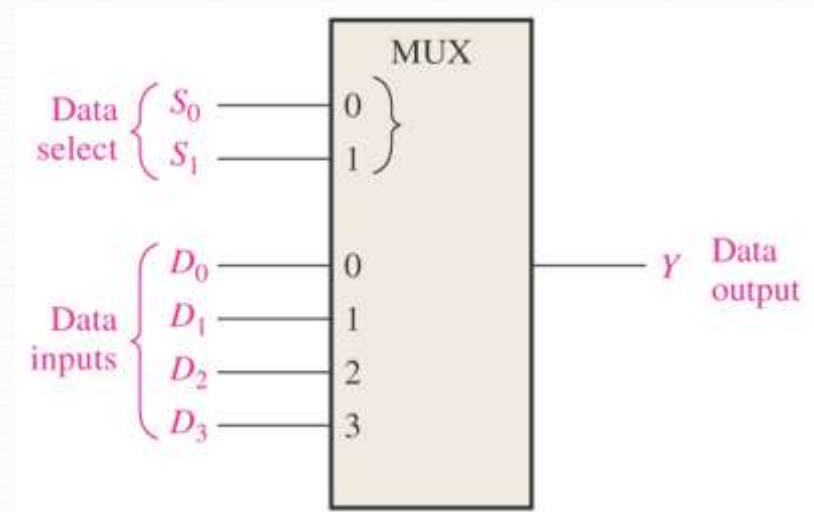
Example: Develop a 16-line-to-4 line encoder using 74LS148s and external logic.

16 inputs; two chips; four outputs



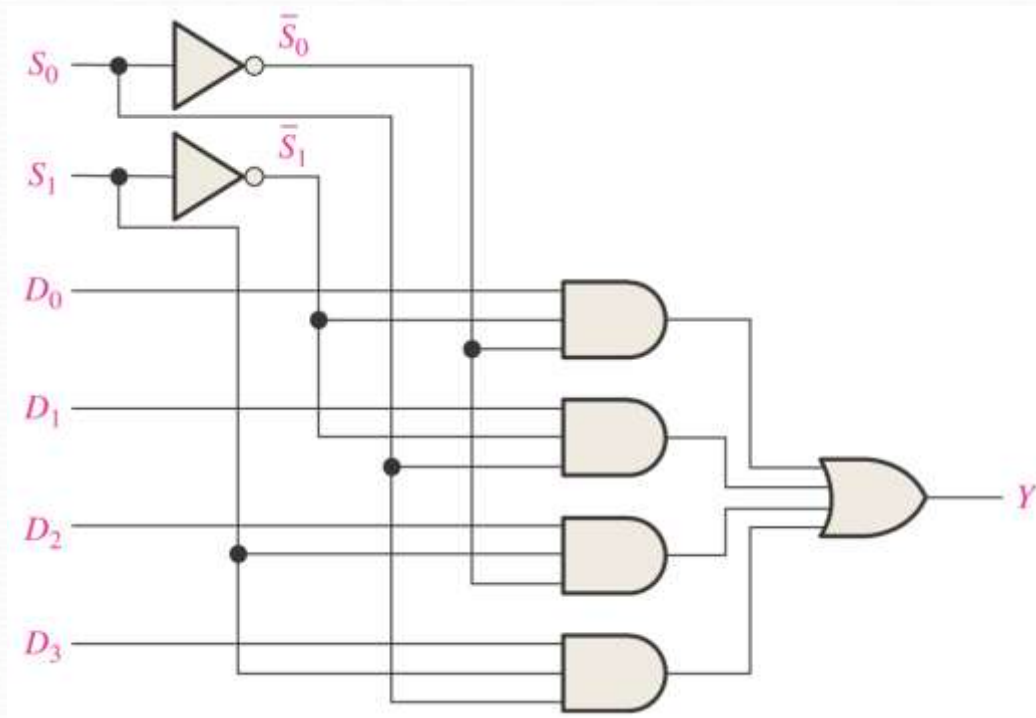
6.7 Multiplexers (Data Selectors)

- Allows digital information from several sources to be routed onto a single line for transmission over that line to a common destination



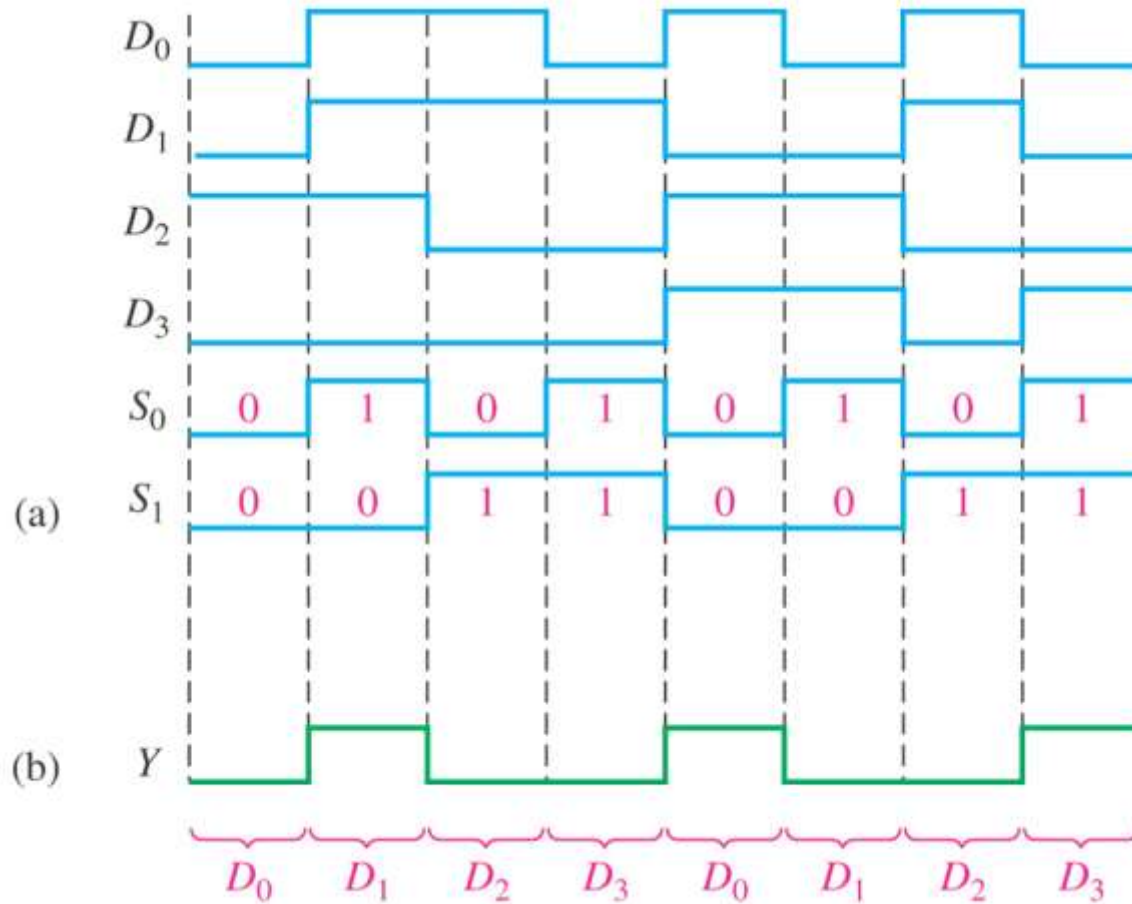
4 选 1 数据选择器功能表

输 入		输 入				输 出
G	S_1	S_0	D_2	D_1	D_0	Y
1	$\times$	$\times$	$\times$	$\times$	$\times$	0
0	0	0	$\times$	$\times$	$\times$	0
			$\times$	$\times$	1	1
	0	1	$\times$	$\times$	0	0
			$\times$	$\times$	1	1
	1	0	$\times$	0	$\times$	0
			$\times$	1	$\times$	1
	1	1	0	$\times$	$\times$	0
			1	$\times$	$\times$	1



$$Y = \bar{S}_1 \bar{S}_0 D_0 + \bar{S}_1 S_0 D_1 + S_1 \bar{S}_0 D_2 + S_1 S_0 D_3$$

Figure 6-48

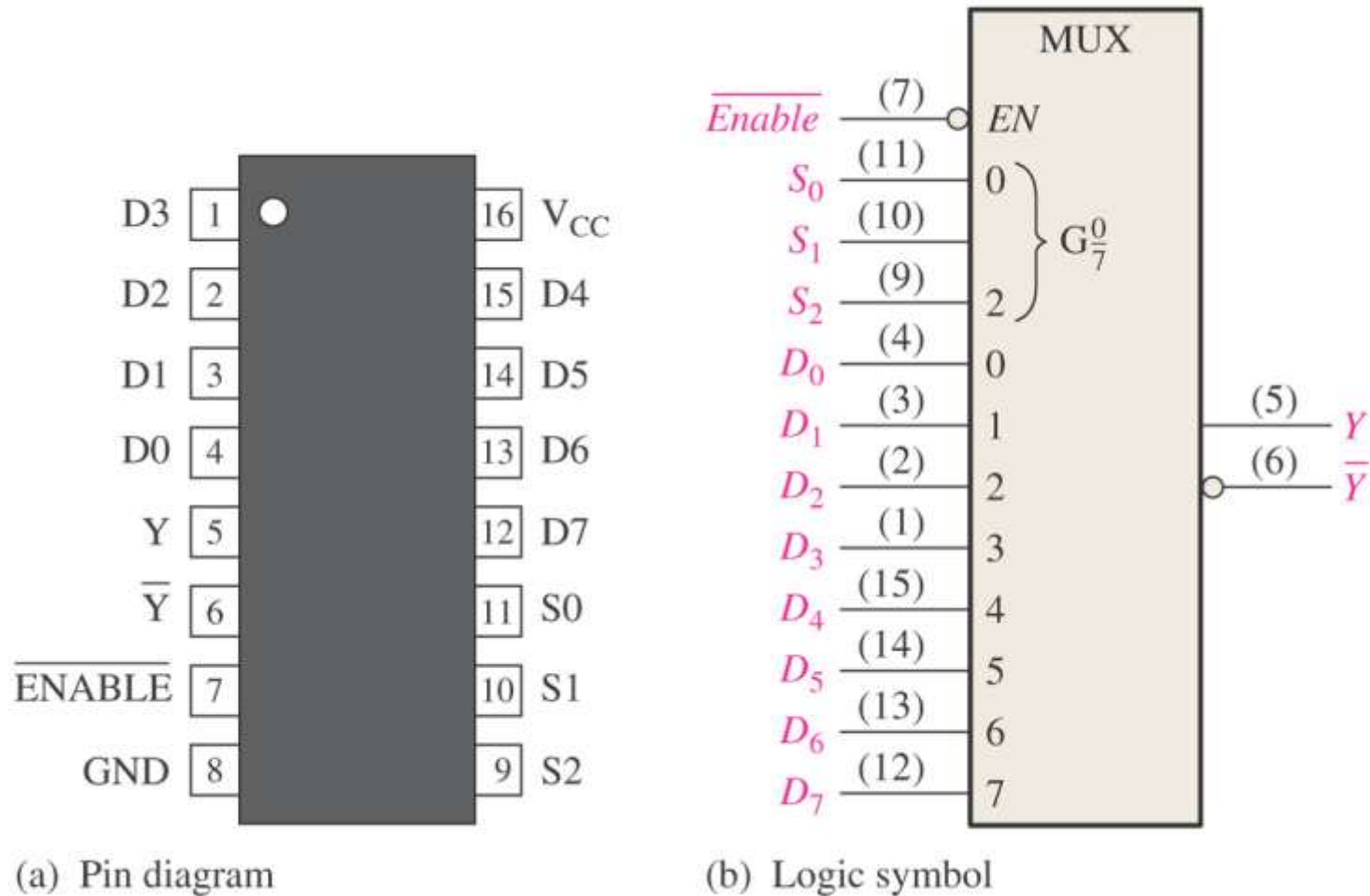


For an 8-input multiplexer

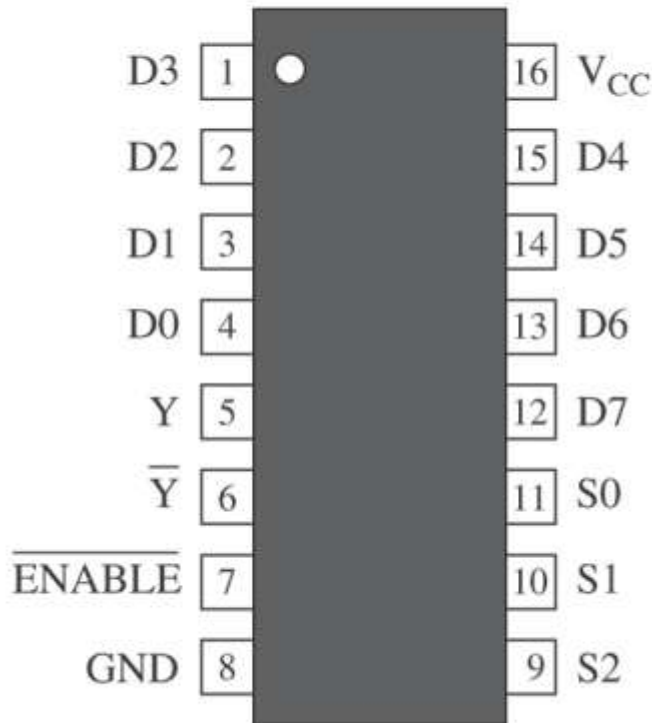
$$Y = (\overline{A_2}\overline{A_1}\overline{A_0})D_0 + (\overline{A_2}\overline{A_1}A_0)D_1 + (\overline{A_2}A_1\overline{A_0})D_2 + (\overline{A_2}A_1A_0)D_3 \\ + (A_2\overline{A_1}\overline{A_0})D_4 + (A_2\overline{A_1}A_0)D_5 + (A_2A_1\overline{A_0})D_6 + (A_2A_1A_0)D_7$$

How to interpret this logic expression?

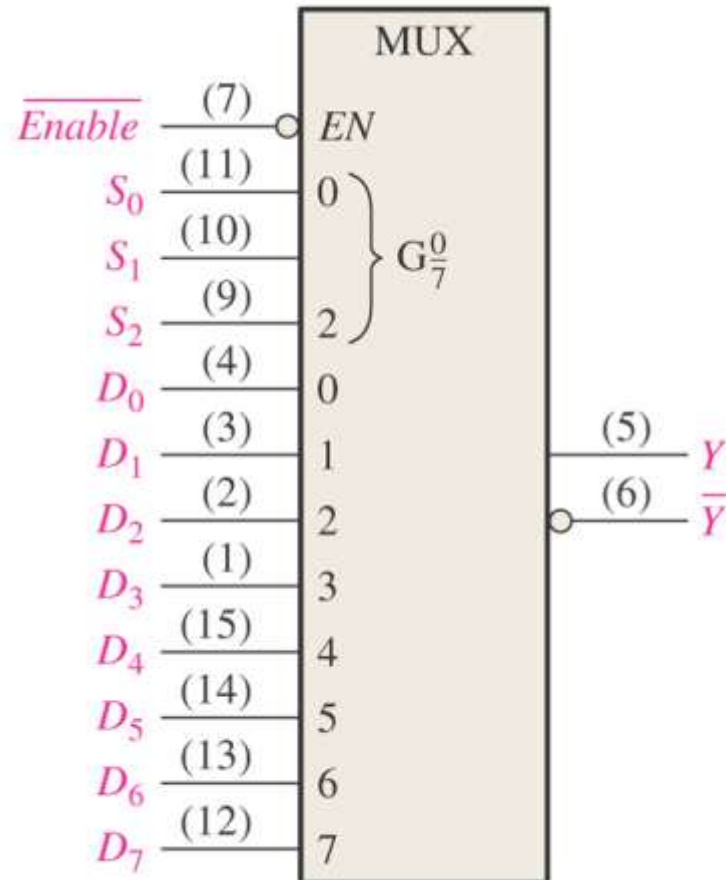
Figure 6–50 Pin diagram and logic symbol for the 74LS151 8-input data selector/multiplexer.



Example: Use 74LS151s and any other logic necessary to multiplex 16 data lines onto a single data-output line.



(a) Pin diagram



(b) Logic symbol

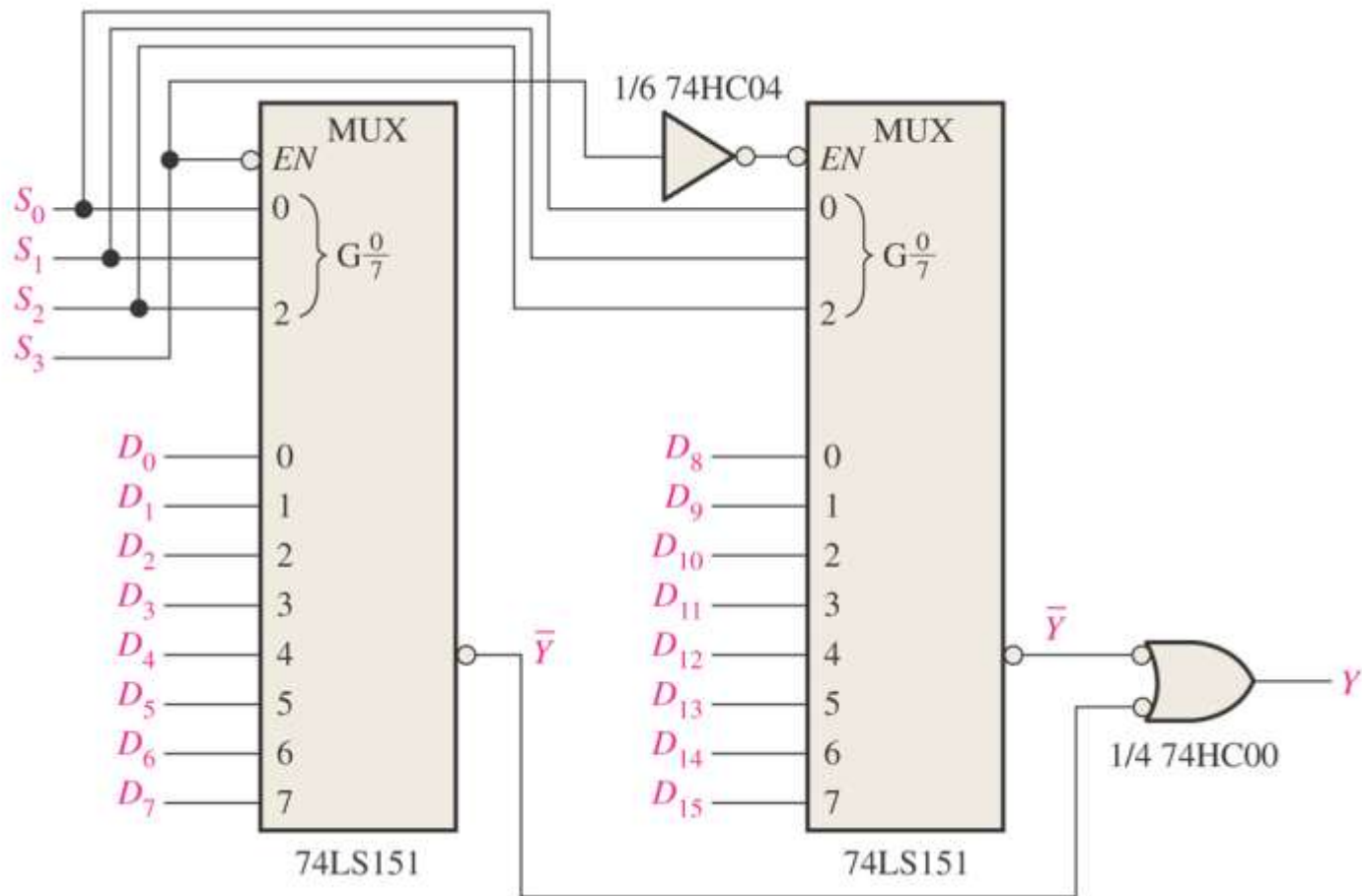
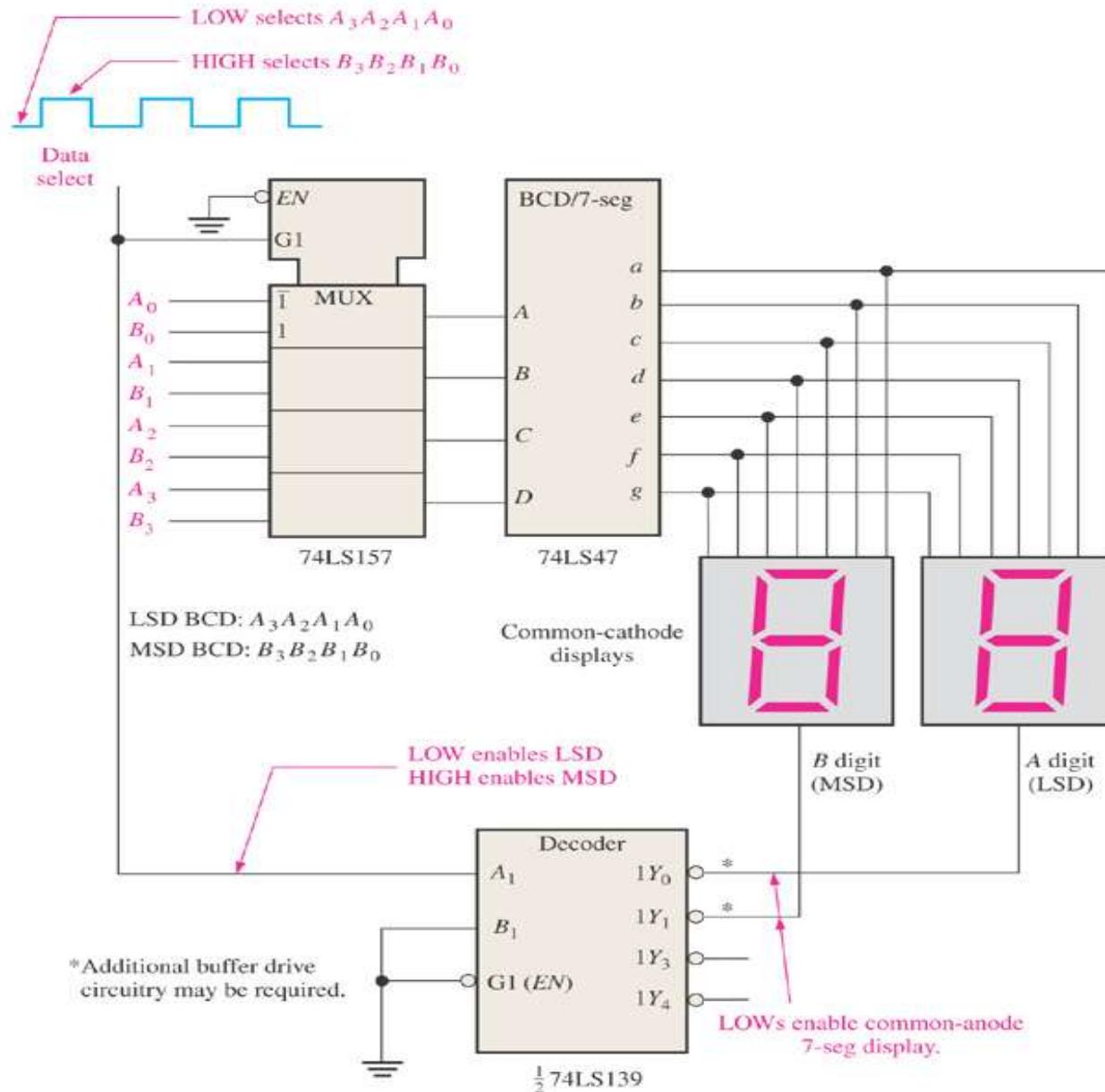
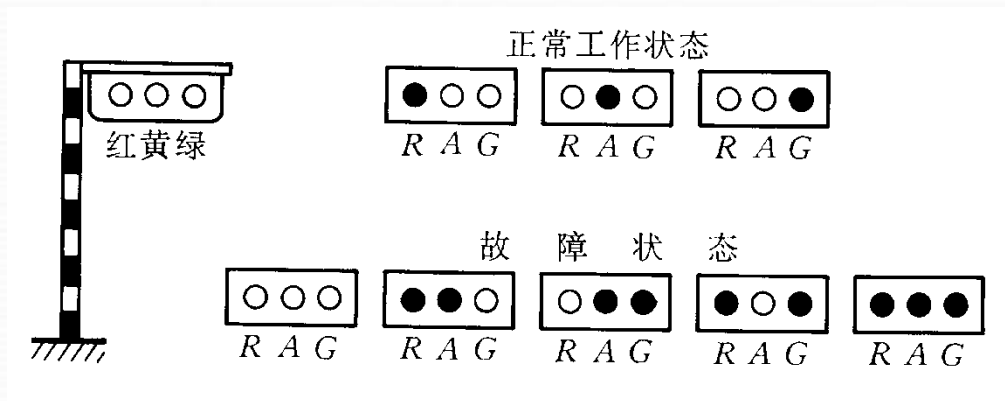


Figure 6–51 A 16-input multiplexer.

Applications: Simplified 7-segment display multiplexing logic.



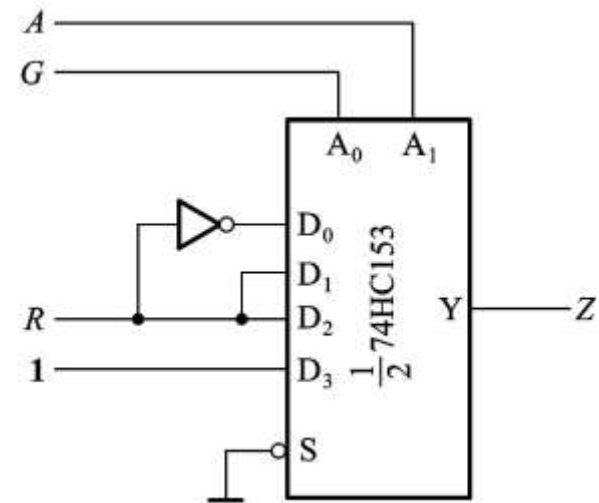
Example:



$$Z = \bar{R}\bar{A}\bar{G} + \bar{R}AG + R\bar{A}\bar{G} + RA\bar{G} + RAG$$

$$= \bar{R}(\bar{A}\bar{G}) + R(\bar{A}\bar{G}) + R(A\bar{G}) + 1 \cdot (AG)$$

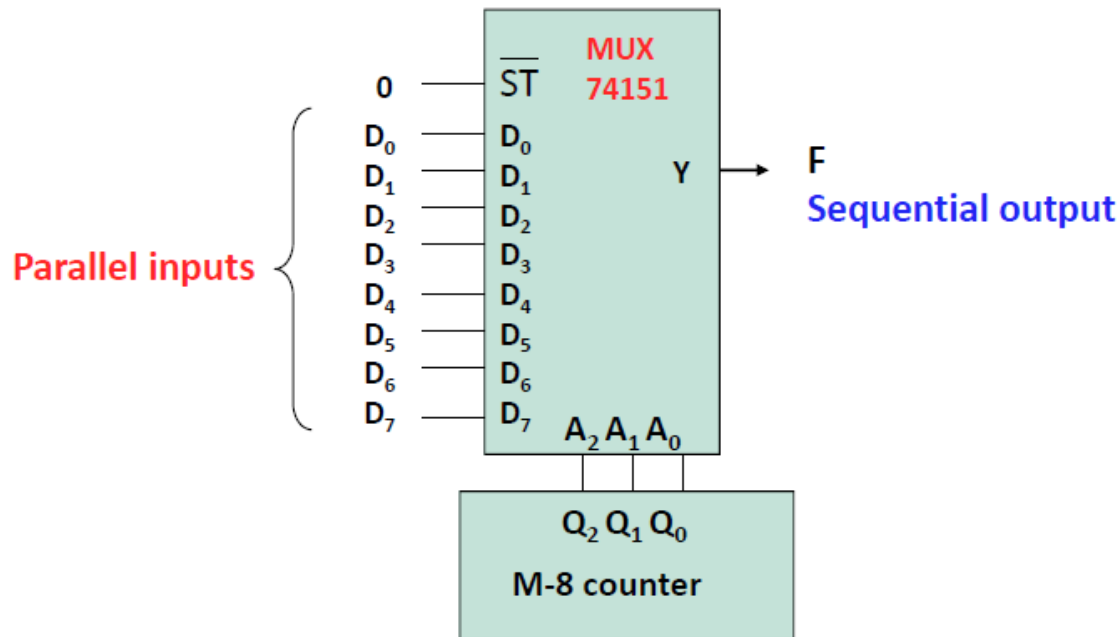
$$Y = D_0(\bar{A}_1\bar{A}_0) + D_1(\bar{A}_1A_0) + D_2(A_1\bar{A}_0) + D_3(A_1A_0)$$



Example:

Parallel-Sequential Transformation 数据的并串转换

MUX + Counter



The state of counter loops over 000 -111, selecting through D_0 - D_7 in sequence for MUX

Example Implement the logic function specified in the following table by using a 74LS151 8-input data **selector/multiplexer**.

Input			Output
A_2	A_1	A_0	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

$$Y = \overline{A_2} \overline{A_1} A_0 + \overline{A_2} A_1 A_0 + A_2 \overline{A_1} A_0 + A_2 A_1 \overline{A_0}$$

From the proceeding truth table, we have the relationship between Y and inputs.

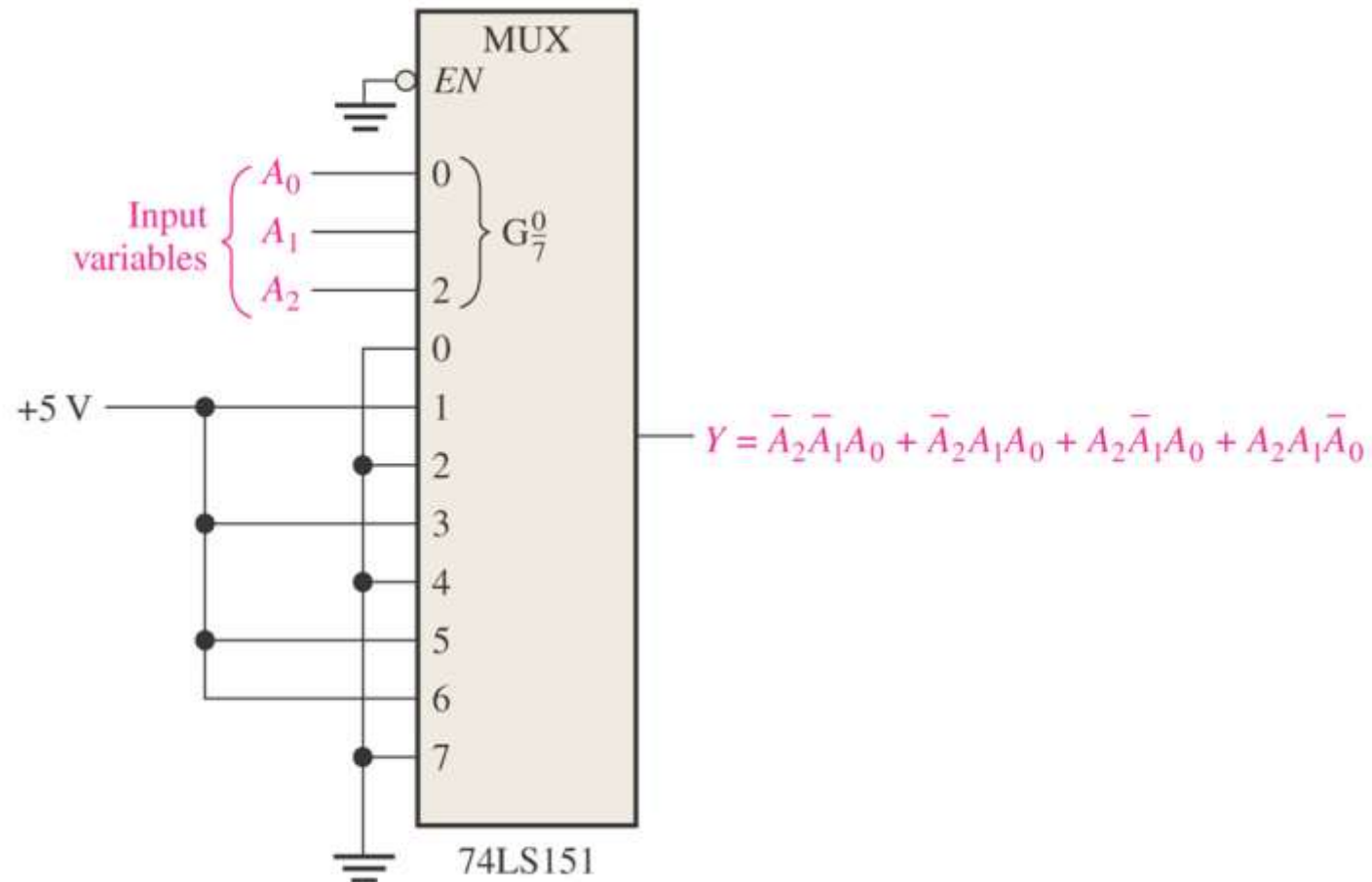
$$Y = \overline{A_2}\overline{A_1}A_0 + \overline{A_2}A_1A_0 + A_2\overline{A_1}A_0 + A_2A_1\overline{A_0}$$

Considering the function of an 8-input multiplexer, the output depends on the inputs in the following way.

$$Y = (\overline{A_2}\overline{A_1}\overline{A_0})D_0 + (\overline{A_2}\overline{A_1}A_0)D_1 + (\overline{A_2}A_1\overline{A_0})D_2 + (\overline{A_2}A_1A_0)D_3 \\ + (A_2\overline{A_1}\overline{A_0})D_4 + (A_2\overline{A_1}A_0)D_5 + (A_2A_1\overline{A_0})D_6 + (A_2A_1A_0)D_7$$

How to implement the first function with the second formula?

Solution



Data selector/multiplexer connected as a 3-variable logic function generator.

6.8 Demultiplexers(解多路复用器/ 数据分配器)

- Reverses the multiplexing function
- Takes digital information from one line and distributes it to a given number of output lines

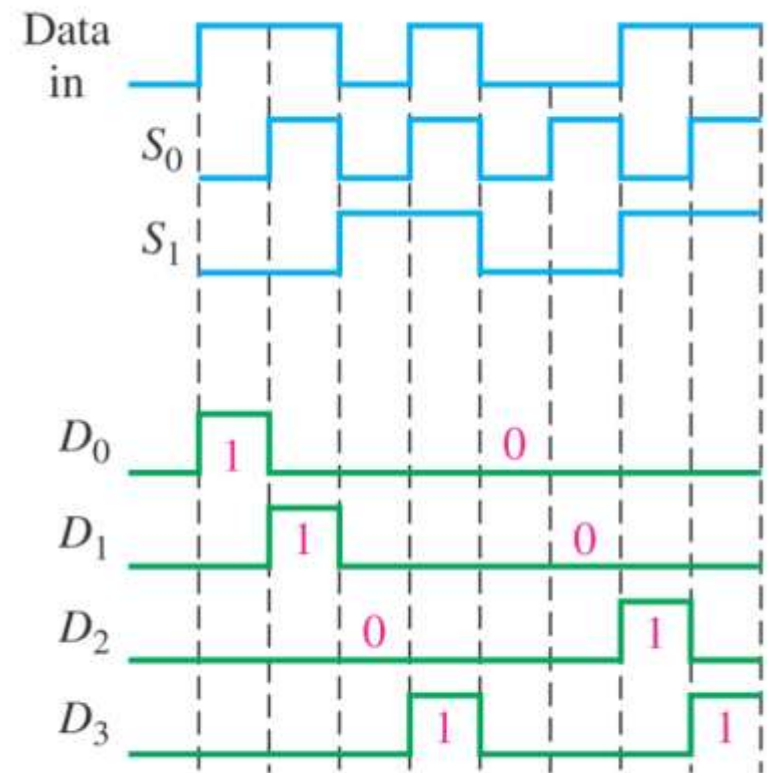
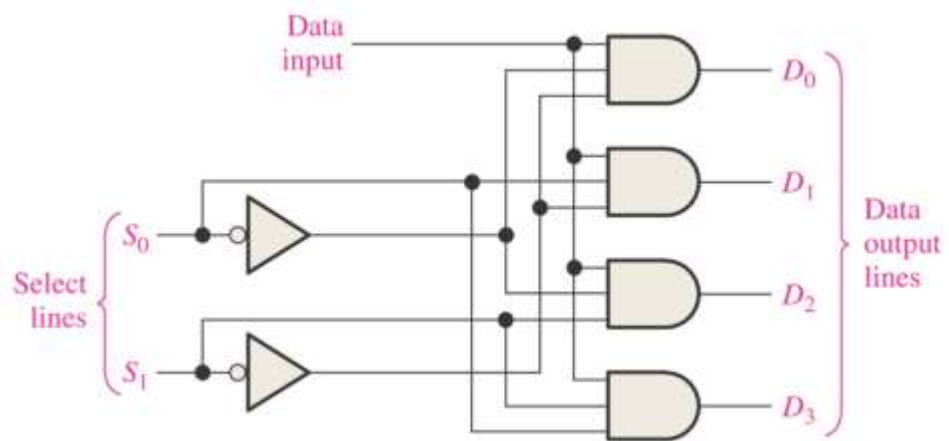


Figure 6–55 A 1-line-to-4-line demultiplexer.

6.9 Race-Hazard of Combinational Logic

组合逻辑电路的竞争冒险

Previous discussions are based on stabilized voltage levels of inputs and outputs. We hereafter discuss the **transient status of circuits given that the inputs switch** from one logic level to another. 前面讨论电路是输入输出处于稳定的逻辑电平的情况。为了保证工作的可靠性，要考虑输入信号逻辑电平发生变化的瞬间电路的工作情况。

In switching of input logic levels, however, the problem of **race-hazard** would arise due to the **transfer delay of logic gates**

竞争：从输入到输出的途径不同，延时时间不同，到达输出的时间不同，这种现象为竞争。

冒险：竞争结果导致逻辑电路产生错误输出，称为冒险或险象。

6.9 Race-Hazard of Combinational Logic

Examples of race-hazard

E.g.1: $F = AB + \bar{A}C$

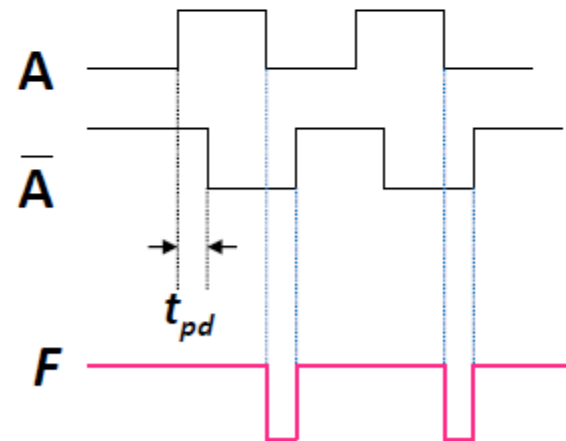
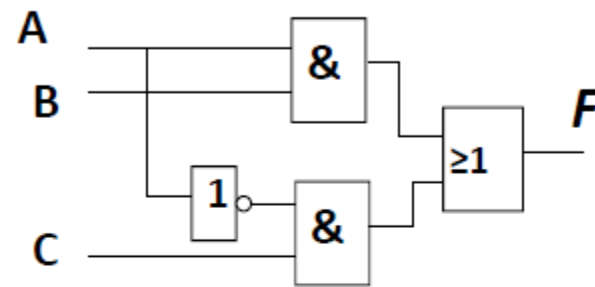
If $B = C = 1$

$F = A + \bar{A} = 1$

F should remain high logic
whatever value A takes

However, a negative glitch is likely
to occur with switching of A , due
to transfer delay of logic gates

hazard (冒险)



glitch (毛刺)

6.9 Race-Hazard of Combinational Logic

E.g.2: $G = (A+B)(\bar{A}+C)$

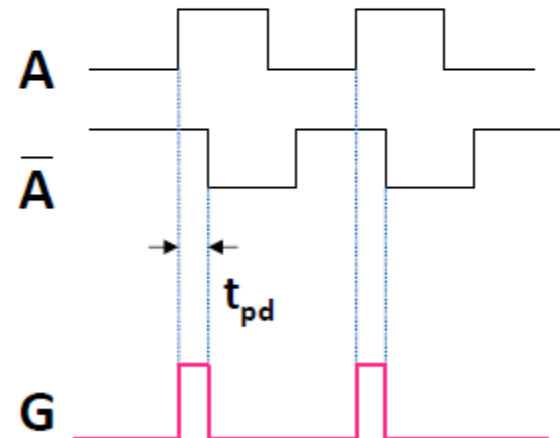
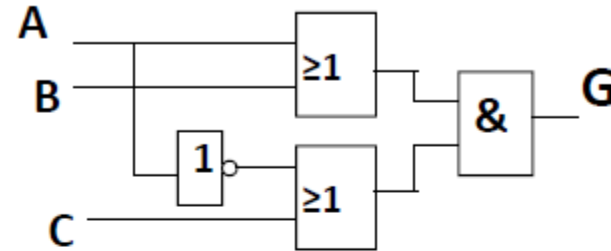
If $B = C = 0$

$G = A \bullet \bar{A} = 0$

G should remain low logic
whatever value A takes

However, a positive glitch is
likely to occur due to transfer
delay of logic gates

hazard (glitch)



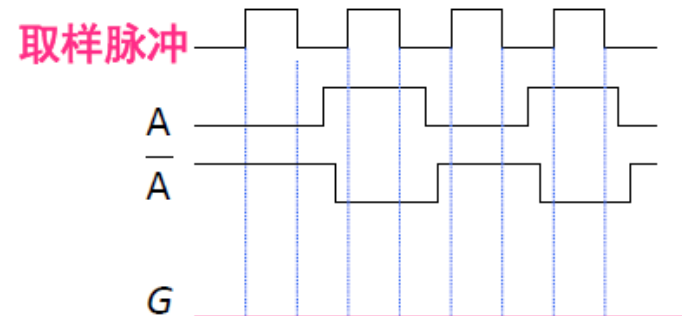
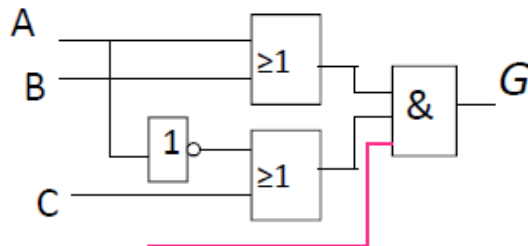
6.9 Race-Hazard of Combinational Logic

Elimination of race-hazard 竞争冒险消除方法

1. Add a filtering capacitor 接入滤波电容

Glitch caused by race-hazard is usually very narrow (10ns), and thus could be filtered by a capacitor connected at the output end.

2. Introduce sample pulses 引入取样脉冲



High logic levels of sample pulses appear when stabilized output could be generated, and thus nullifies the glitch in its zero periods 在输出端接取样脉冲，仅在输出处于稳定值时出现。取样脉冲为0期间，输出端信息无效。

6.9 Race-Hazard of Combinational Logic

3. Modify logic function

$$\begin{array}{l} F = A + \bar{A} \\ F = A \bullet \bar{A} \end{array} \quad \left. \vphantom{\begin{array}{l} F = A + \bar{A} \\ F = A \bullet \bar{A} \end{array}} \right\} \text{Harzard}$$

We could introduce redundancy to avoid harzard
引入冗余项，可以消除冒险

$$F = AB + \bar{A}C \quad \Longrightarrow \quad F = AB + \bar{A}C + BC$$

↓
No harzard

Summary

- Some widely used combinational logics
 - basic adders, parallel binary adders, ripple carry versus look-ahead carry adders, comparators, decoders, encoders, code converters, multiplexers, demultiplexers
- How to implement logic with decoders and multiplexers?
 - Methods

HW

- 10th 227-230

- 14
- 20
- 28
- 29

- 11th 214-216

- 14
- 20
- 28
- 29